

# INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN: AN AGILE, ITERATIVE APPROACH

SATZINGER | JACKSON | BURD

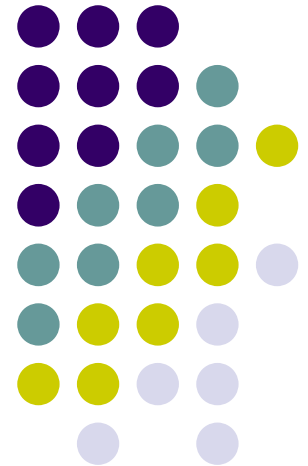
## CHAPTER 4

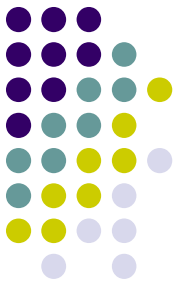
# Domain Modeling

## Chapter 4

Introduction to Systems  
Analysis and Design:  
An Agile, Iterative Approach  
6<sup>th</sup> Ed

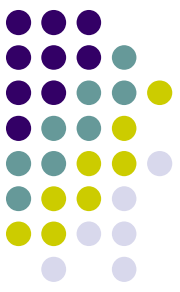
Satzinger, Jackson & Burd





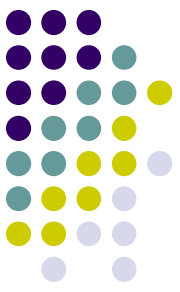
# Chapter 4 Outline

- “Things” in the Problem Domain （问题域中的“事物”）
  - Data entities （数据实体）
  - Domain classes （域类）
- The Entity-Relationship Diagram （实体 - 关系图）
- The Domain Model Class Diagram （域模型类图）
- The Domain Model Class Diagram in the RMO example （ RMO 示例中的域模型类图）



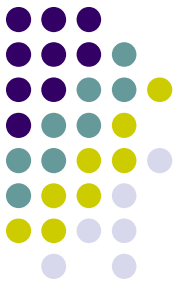
# Learning Objectives

- Explain how the concept of “things” in the problem domain also define requirements (解释问题域中的“事物”概念如何定义需求)
- Identify and analyze data entities (数据实体) and domain classes (域类) needed in the system
- Read, interpret, and create an entity-relationship diagram (实体 - 关系图)
- Read, interpret, and create a domain model class diagram (域模型类图)
- Understand the domain model class diagram for the RMO Consolidated Sales and Marketing System



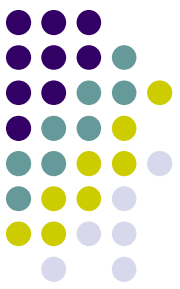
# Overview

- Chapter 3 provided an overview of identifying use cases to define functional requirements (第三章描述了如何识别用例来定义功能需求)
- This chapter focuses on another key concepts for defining requirements— data entities (数据实体) or domain classes (域类)
  - In the RMO Tradeshow System from Chapter 1, some domain classes are Supplier, Product, and Contact
  - In this chapter's opening case Waiters on Call, examples of domain classes are Restaurants, Menu items, Customers, Orders, Drivers, Addresses, Routes, and Payments



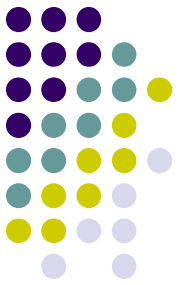
# Chapter 4 Outline

- “Things” in the Problem Domain (问题域中的“事物”)
  - Data entities
  - Domain classes
- The Entity-Relationship Diagram
- The Domain Model Class Diagram
- The Domain Model Class Diagram in the RMO example



# Things in the Problem Domain

- Problem domain (问题域) — the specific area (or domain) of the users' business need that is within the scope of the new system (在新系统范围内的用户业务需求的特定区域 / 领域) .
- “Things” are those items users work with when accomplishing tasks that need to be remembered (“事物”是用户在完成任务时使用的项目)
  - Examples of “Things” are products, sales, shippers, customers, invoices, payments, etc. (例如：产品、销售、发货人、客户、发票、付款等)



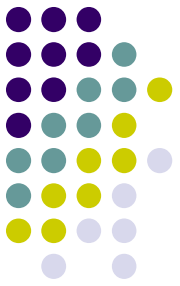
# Things in the Problem Domain

- These “Things” are modeled as domain classes or data entities （这些“事物”被建模为域类或数据实体）
- In this course, we will call them **domain classes** (域类). In database class you call them **data entities** (数据实体)



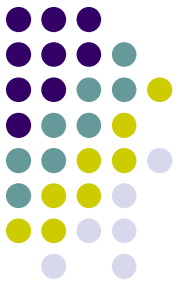
# Things in the Problem Domain

## Two Techniques for Identifying them



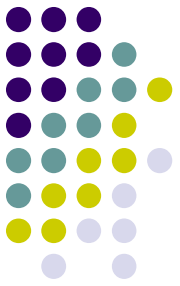
- Brainstorming Technique (头脑风暴法)
  - Use a checklist of all of the usual types of things typically found and brainstorm to identify domain classes of each type (使用所有事物类型的清单)
- Noun Technique (名词法)
  - Identify all of the nouns that come up when the system is described and determine if each is a domain class, an attribute, or not something we need to remember (识别在描述系统时出现的所有名词)

# Brainstorming Technique: Steps (头脑风暴法)



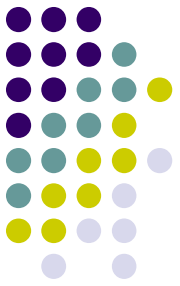
1. Identify a user (用户) and a set of use cases (用例)
2. Brainstorm with the user to identify things involved when carrying out the use case (定义当实施用例时所包含的事物) — that is, things about which information should be captured by the system.
3. Use the types of things (categories) to systematically ask questions about potential things, such as the following: Are there any tangible things you store information about? Are there any locations involved? Are there roles played by people that you need to remember?  
(使用事物类型来系统地询问关于潜在事物的问题)

# Brainstorming Technique: Steps (头脑风暴法)



4. Continue to work with all types of users and stakeholders to expand the brainstorming list (继续与各种类型的用户和利益相关者工作，以此来扩大头脑风暴列表)
5. Merge the results, eliminate any duplicates, and compile an initial list (合并所有的结果，排除所偶的重复内容并编制一个初步列表)

# Brainstorming Technique: Example

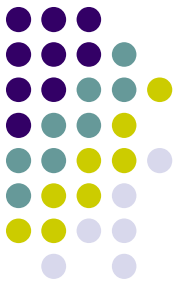


当你在与航空公司的地勤人员沟通，来帮助完善机场航班规划系统时，他提到：

➤ 1&2. 根据用例来找出系统中的“Things”，即事物

“作为地勤员工，我们需要在每一架飞机起飞前，安排检修人员对飞机进行全方位的检查，检查至少应当在航班登机时间开始前半小时完成。”

# Brainstorming Technique: Example

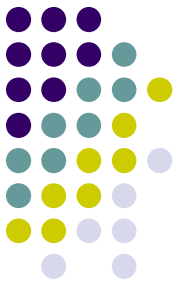


当你在与航空公司的地勤人员沟通，来帮助完善机场航班规划系统时，他提到：

➤ 1&2. 根据用例来找出系统中的“Things”，即事物

“作为地勤员工，我们需要在每一架飞机起飞前，安排检修人员对飞机进行全方位的检查，检查至少应当在航班登机时间开始前半小时完成。”

# Brainstorming Technique: Example

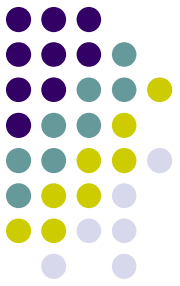


当你在与航空公司的地勤人员沟通，来帮助完善机场航班规划系统时，他提到：

➤ 3. 根据事物来系统性地询问关于潜在事物的问题。

“作为地勤员工，我们需要在每一架飞机起飞前，安排检修人员对飞机进行全方位的检查，检查至少应当在航班登机时间开始前半小时完成。”

# Brainstorming Technique: Example



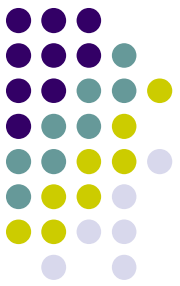
当你在与航空公司的地勤人员沟通，来帮助完善机场航班规划系统时，他提到：

- 3. 根据事物来系统性地询问关于潜在事物的问题。（小 Tips：找动词）

“作为地勤员工，我们需要在每一架飞机起飞前，安排检修人员对飞机进行全方位的检查，检查至少应当在航班登机时间开始前半小时完成。”

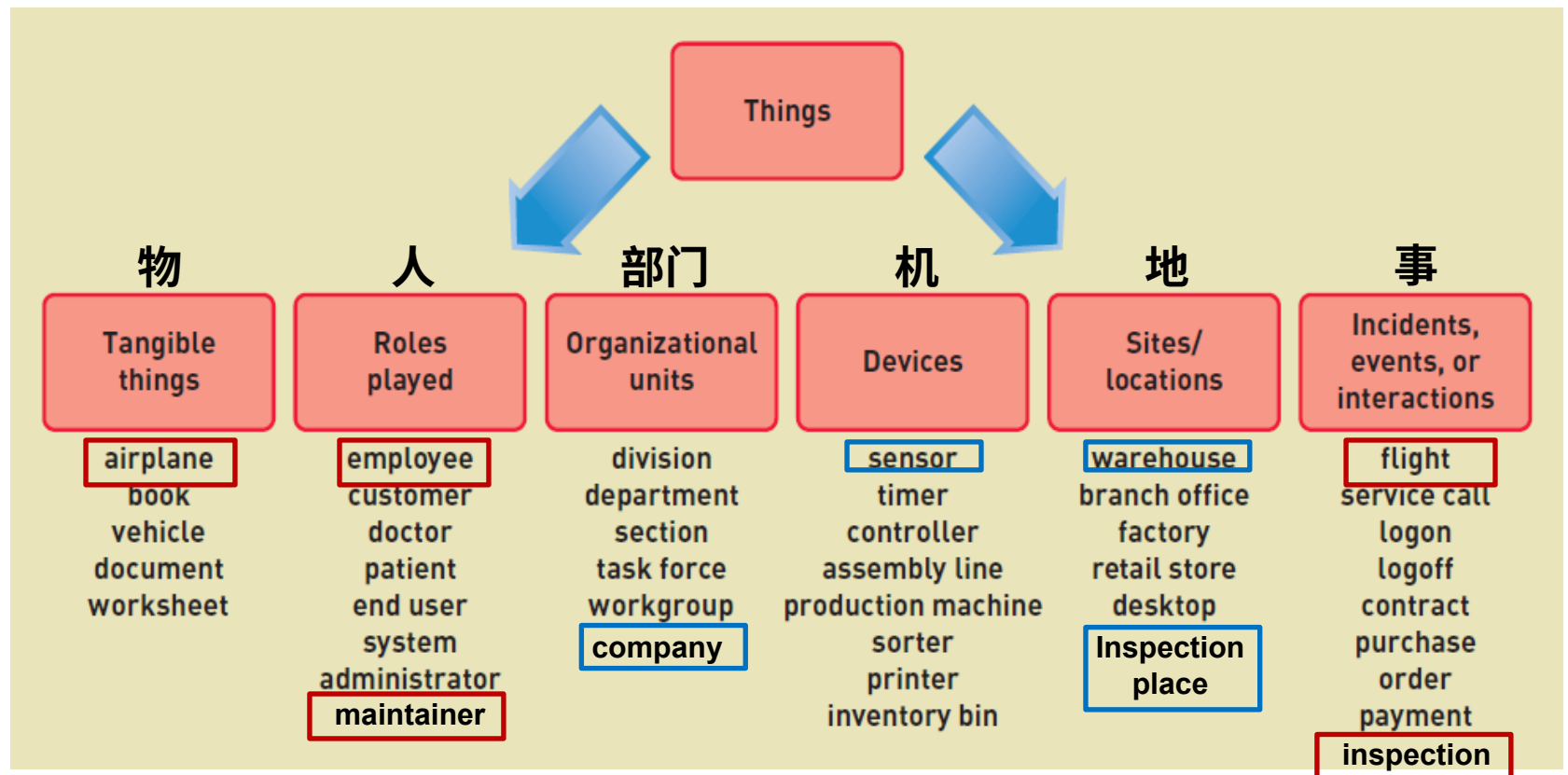
潜在事物：  
在哪里做检查？  
用什么设备进行检查？  
这些设备存放在哪里？

潜在事物：  
哪家公司？  
机长是谁？  
乘务员包括谁？



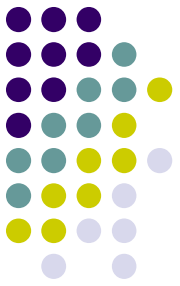
# Brainstorming Technique

- 事物（飞机；地勤员工；检修人员；航班；检查）
- 潜在事物（存放仓库；检查地点；检查装置；公司；机长；乘务员）



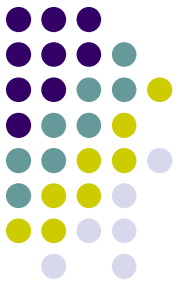


# The Noun Technique (名词技术)



- A technique to identify problem domain classes (things) by finding, classifying, and refining a list of nouns that come up in discussions or documents (通过查找、分类和精炼名词来识别问题域类的技术)
- Popular technique. Systematic.
- Good place to start when there are no users available to help brainstorm (当没有用户可以进行头脑风暴时，是个很好的方式)

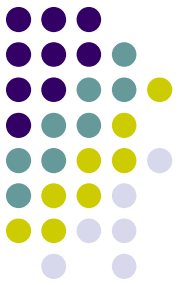
# The Noun Technique: Steps



1. Using the use cases, actors, and other information about the system—including inputs and outputs—identify all nouns. (通过用例、参与者和其他系统信息来确定所有名词)
  - For the RMO CSMS, the nouns might include customer, product item, sale, confirmation, transaction, shipping, bank, change request, summary report, management, transaction report, accounting, back order, back order notification, return, return confirmation... (对 RMO 来说, 名词包括客户、产品、销售、交易、运输、银行、变更请求、汇总报告、管理、交易报告、会计、退货、退货通知、退货确认)
2. Using other information from existing systems, current procedures, and current reports or forms, add items or categories of information needed. (通过现有系统的其他信息, 例如当前过程、报告或表格来添加)
  - For the RMO CSMS, these might include price, size, color, style, season, inventory quantity, payment method, and shipping address. (对于 RMO csm, 这些可能包括价格、尺寸、颜色、风格、季节、库存数量、付款方式和送货地址。)

# The Noun Technique:

## Steps (continued)



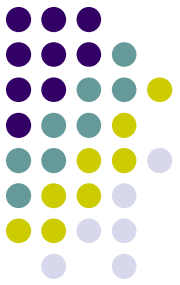
3. As this list of nouns builds, refine it. Ask these questions about each noun to help you decide whether you should include it : （针对每个名词问下面这些问题来决定是否应该保留）
- Is it a unique thing the system needs to know about? （这是系统需要了解的独特的事物吗？）
  - Is it inside the scope of the system I am working on? （这个事物是在我处理的系统范围之内吗？）

Ask these questions to decide to exclude it （问下面这些问题能帮助你决定是否应该排除）：

- Is it really a synonym for some other thing I have identified? （这真的是我已经确认的其他事物的同义词吗？）
- Is it really just an output of the system produced from other information I have identified? （是否是我定义的其他系统的输出？）
- Is it really just an input that results in recording some other information I have identified? （是否是我定义的其他信息的输入？）

# The Noun Technique:

## Steps (continued)



Ask these questions to research it （问一下这些问题能帮助你决定是否应该研究它）：

- Is it likely to be a specific piece of information (attribute) about some other thing I have identified? （这像是我已经确定的事务信息的属性吗？）
- Is it something I might need if assumptions change? （如果属性变了我会需要这些事物吗？）

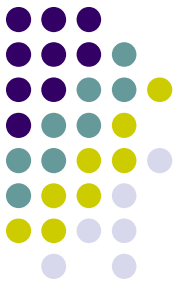
4. Create a master list of all nouns identified and then note whether each one should be included, excluded, or researched further. （创建所有已确认名词的主要列表，然后注意每个名词是否要保留、排除或是研究。）

5. Review the list with users, stakeholders, and team members and then define the list of things in the problem domain. （与用户、利益相关者和团队成员一起评审，然后修改问题域中的事物列表。）

# Partial List of Nouns for RMO

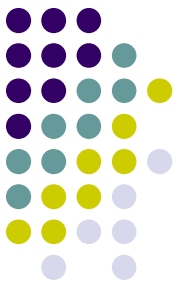
With notes on  
whether to  
include as  
domain class

| Identified noun          | Notes on including noun as a thing to store                             |
|--------------------------|-------------------------------------------------------------------------|
| Accounting               | We know who they are. No need to store it.                              |
| Back order               | A special type of order? Or a value of order status? Research.          |
| Back-order information   | An output that can be produced from other information.                  |
| Bank                     | Only one of them. No need to store.                                     |
| Catalog                  | Yes, need to recall them, for different seasons and years. Include.     |
| Catalog activity reports | An output that can be produced from other information. Not stored.      |
| Catalog details          | Same as catalog? Or the same as product items in the catalog? Research. |
| Change request           | An input resulting in remembering changes to an order.                  |
| Charge adjustment        | An input resulting in a transaction.                                    |
| Color                    | One piece of information about a product item.                          |
| Confirmation             | An output produced from other information. Not stored.                  |
| Credit card information  | Part of an order? Or part of customer information? Research.            |
| Customer                 | Yes, a key thing with lots of details required. Include.                |
| Customer account         | Possibly required if an RMO payment plan is included. Research.         |
| Fulfillment reports      | An output produced from information about shipments. Not stored.        |
| Inventory quantity       | One piece of information about a product item. Research.                |
| Management               | We know who they are. No need to store.                                 |
| Marketing                | We know who they are. No need to store.                                 |
| Merchandising            | We know who they are. No need to store.                                 |



# Details about Domain Classes

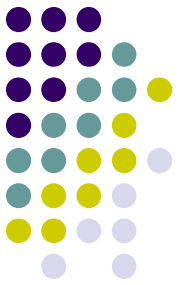
- Attribute (属性) — describes one piece of information about each instance of the class (描述类的每个实例的一条信息)
  - Customer has first name, last name, phone number
- Identifier or key (标识符 / 键)
  - One attribute uniquely identifies an instance of the class (唯一地标识类的实例) . Required for data entities, optional for domain classes. Customer ID identifies a customer
- Compound attribute (复合属性)
  - Two or more attributes combined into one structure (两个或多个属性组合成一个结构以简化模型) to simplify the model. (E.g., address rather than including number, street, city, state, zip separately). Sometimes an identifier or key is a compound attribute.



# Attributes and Values

- Class is a type of thing. (类是一种“事物”类型)
- Object is a specific instance of the class. Each instance has its own values for an attribute (对象是类的特定实例。每个实例都有自己的属性值)

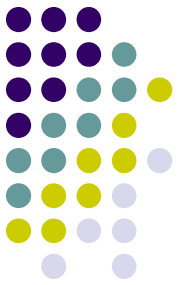
| All customers have these attributes: | Each customer has a value for each attribute: |          |          |
|--------------------------------------|-----------------------------------------------|----------|----------|
| Customer ID                          | 101                                           | 102      | 103      |
| First name                           | John                                          | Mary     | Bill     |
| Last name                            | Smith                                         | Jones    | Casper   |
| Home phone                           | 555-9182                                      | 423-1298 | 874-1297 |
| Work phone                           | 555-3425                                      | 423-3419 | 874-8546 |



# Chapter 4 Outline

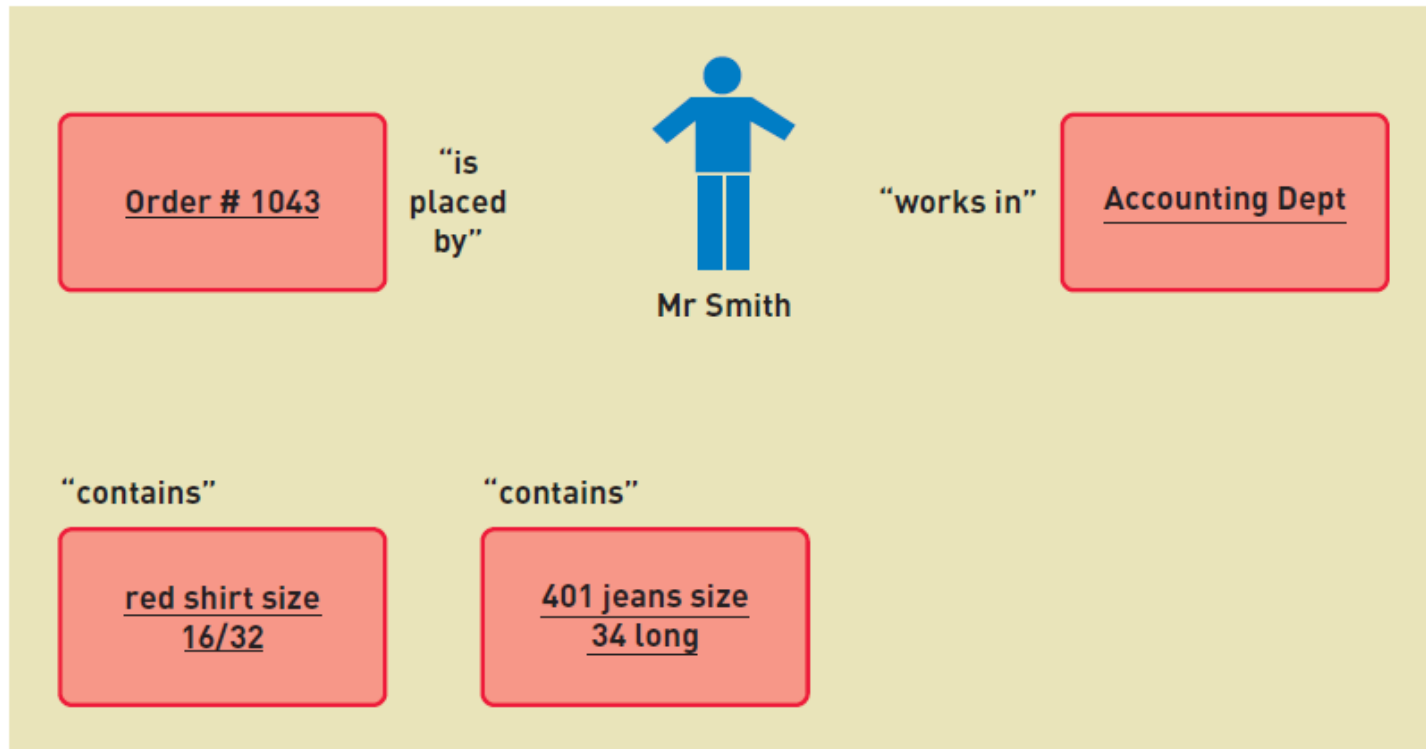
- “Things” in the Problem Domain
  - Data entities
  - Domain classes
- The Entity-Relationship Diagram (实体 - 关系图)
- The Domain Model Class Diagram
- The Domain Model Class Diagram in the RMO example



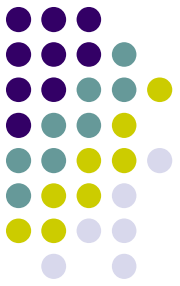


# Associations Among Things

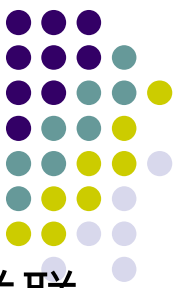
- Association (关联) — a naturally occurring relationship between classes (类之间自然发生的关系)



# Just to Clarify...



- Called **association** (关联) on class diagram in UML
  - Multiplicity (重数) is term for the number of associations between classes (类) : 1 to 1 or 1 to many
  - We are emphasizing UML in this text
- Called **relationship** (关系) on ERD in database class
  - Cardinality (基数) is term for number of relationships in entity relationship diagrams (实体关系图) : 1 to 1 or 1 to many
- Associations and Relationships apply in two directions (关联和关系是双向的)
  - A customer places an order (客户下订单)
  - An order is placed by a customer (订单是由客户下的)



# Minimum and Maximum Multiplicity

- Associations have **minimum** and **maximum** constraints (关联具有最小和最大约束)
  - minimum is zero, the association is optional (可选的)
  - If minimum is at least one, the association is mandatory (强制的)

Mr. Jones has placed no order yet, but there might be many placed over time.

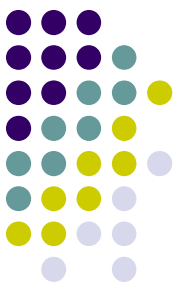
multiplicity/cardinality is zero or more—optional relationship

A particular order is placed by Mr. Smith. There can't be an order without stating who the customer is.

multiplicity/cardinality is one and only one—mandatory relationship

An order contains at least one item, but it could contain many items.

multiplicity/cardinality is one or more—mandatory relationship

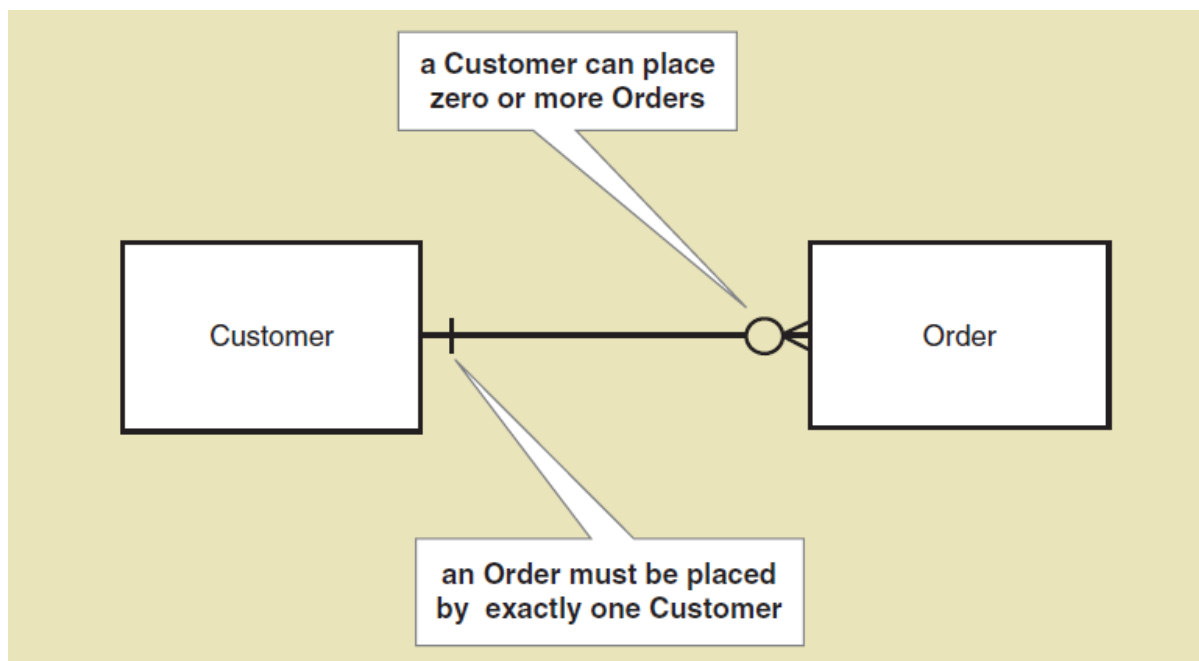


# Types of Associations

- Binary Association (二元关系)
  - Associations between exactly two different classes (两种不同事物之间的关系)
    - Course Section includes Students (课程组包括学生)
    - Members join Club (会员加入俱乐部)
- Unary Association (recursive) (一元关系 / 回归关系)
  - Associations between two instances of the same class (同一件类型的两件不同事物间的关系)
    - Person married to person (人与人结婚)
    - Part is made using parts (零件是由零件构成的)
- Ternary Association (three) (三元关系)
- N-ary Association (between n) (N元关系)

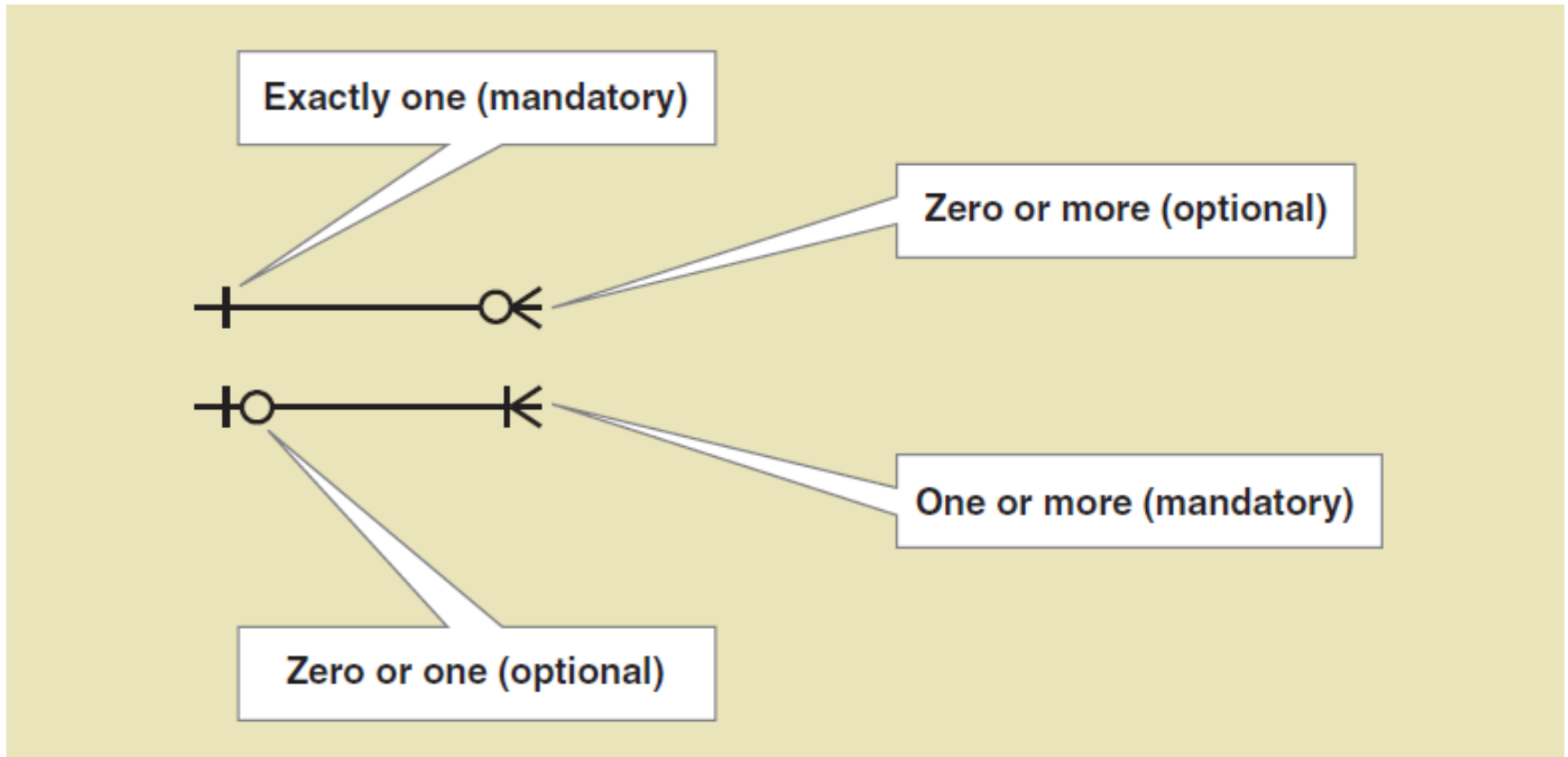
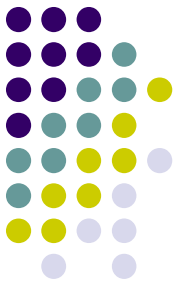
# Example of ERD Notation

- Entity-Relationship Diagrams : ERD (实体关系图)
- A simple ERD without showing attributes
  - 矩形：数据实体
  - 连接矩形的直线：数据实体之间的联系



# ERD Cardinality Symbols

often called the crows feet notation (鱼尾纹计数法)



# Semantic Net

Shows instances and how they are linked (显示实例及其链接方式)

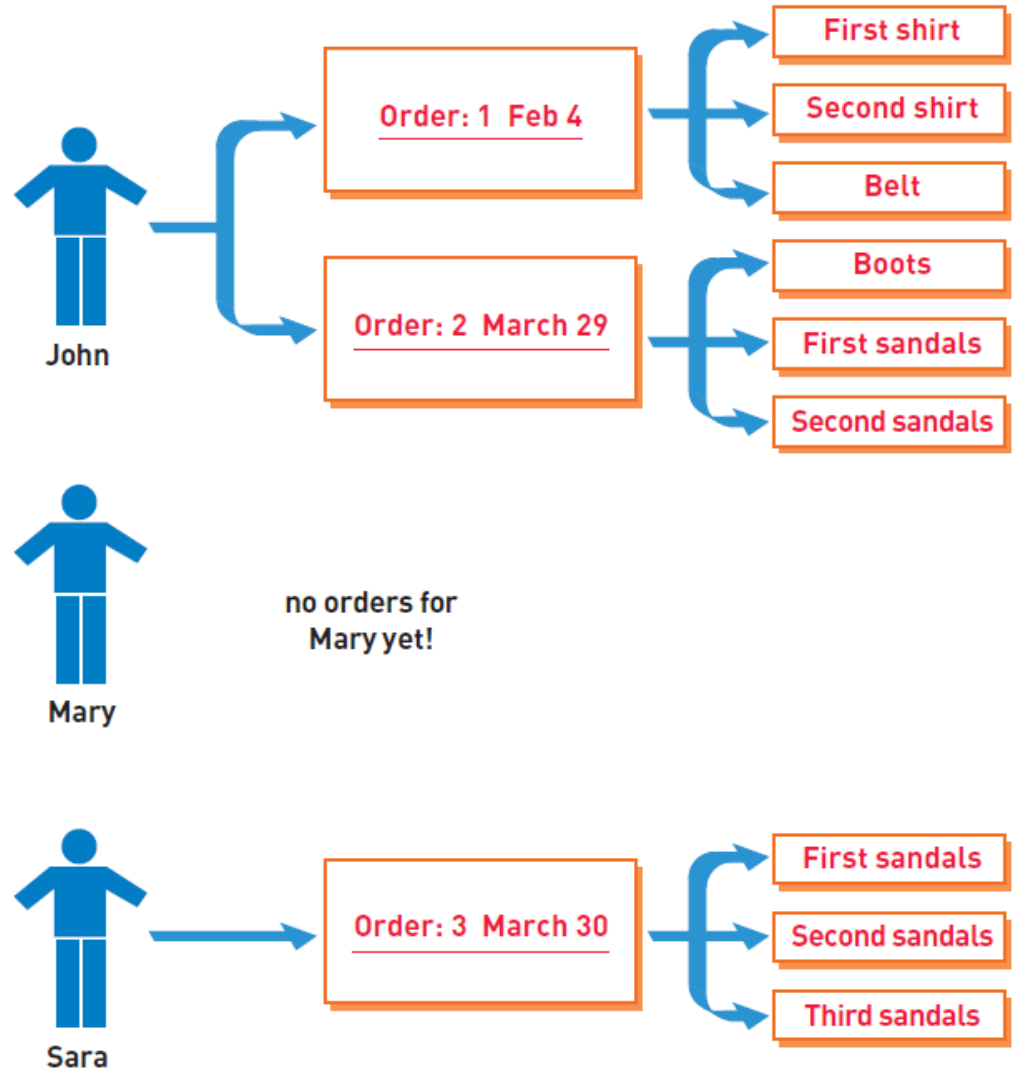
Example shows instances of three classes (示例展示了三个类的实例)

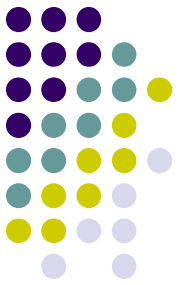
Quick quiz:

How many associations are there?

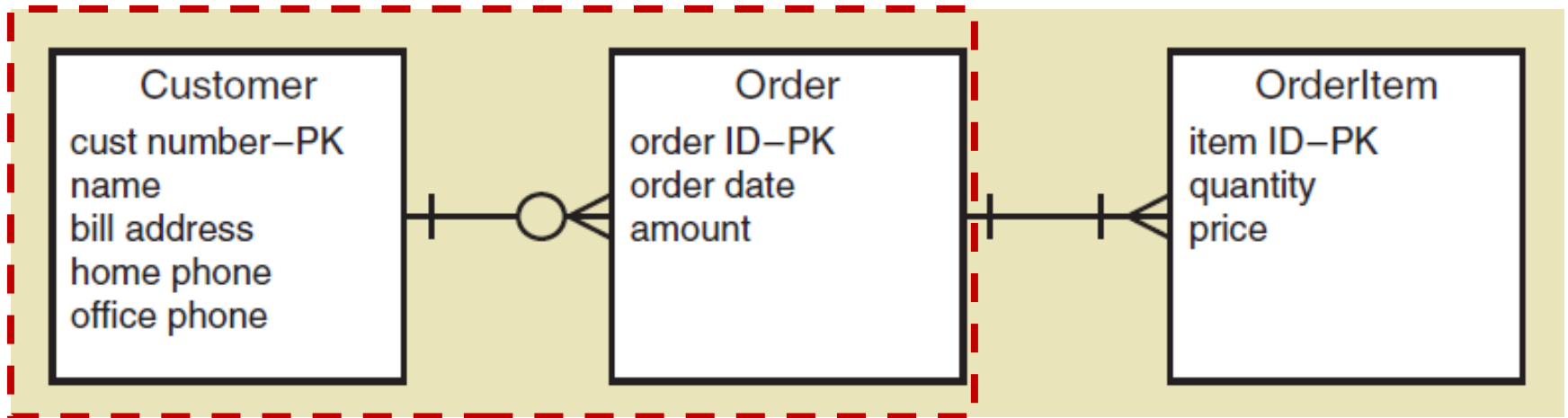
What are the minimum and maximum multiplicities in each direction?

What type of associations are they?

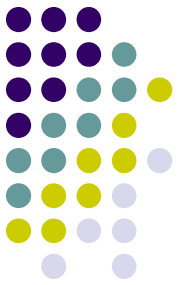




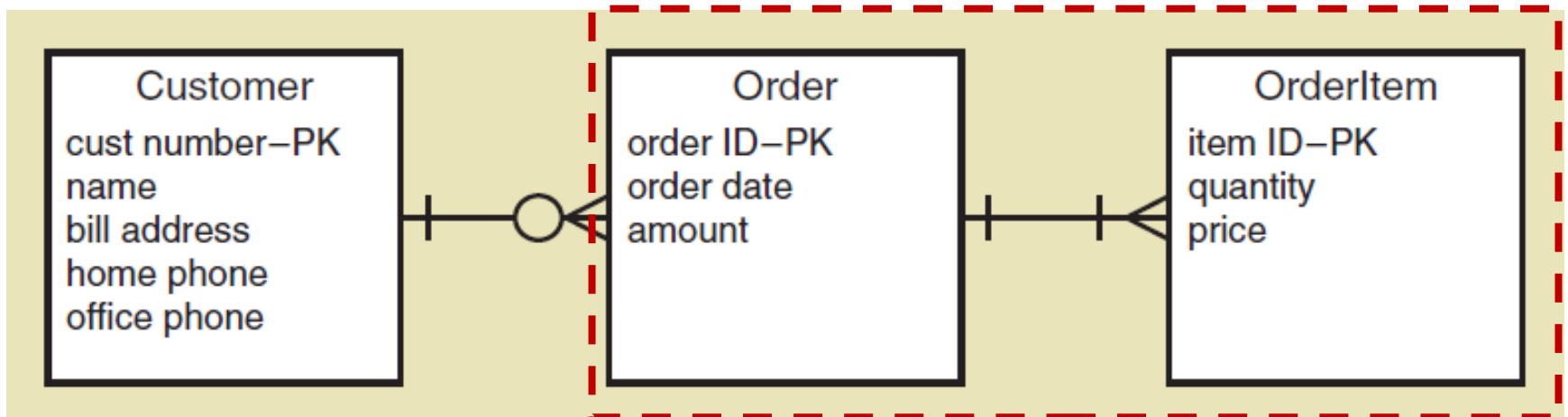
# Expanded ERD with Attributes

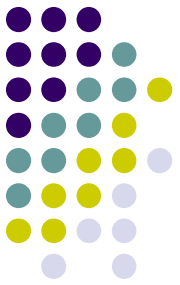




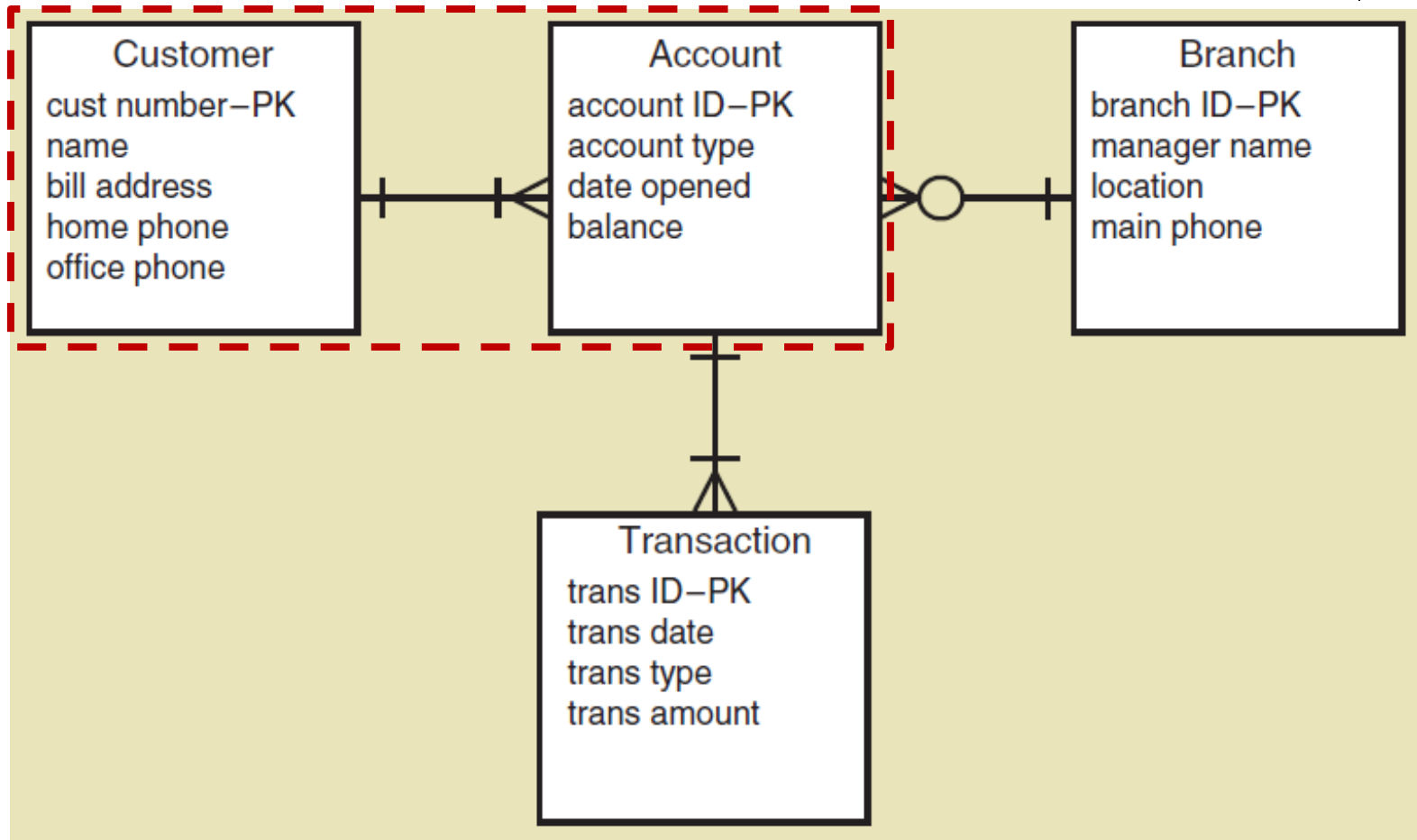


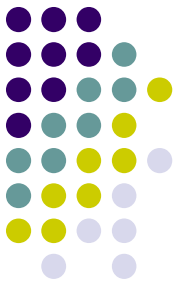
# Expanded ERD with Attributes



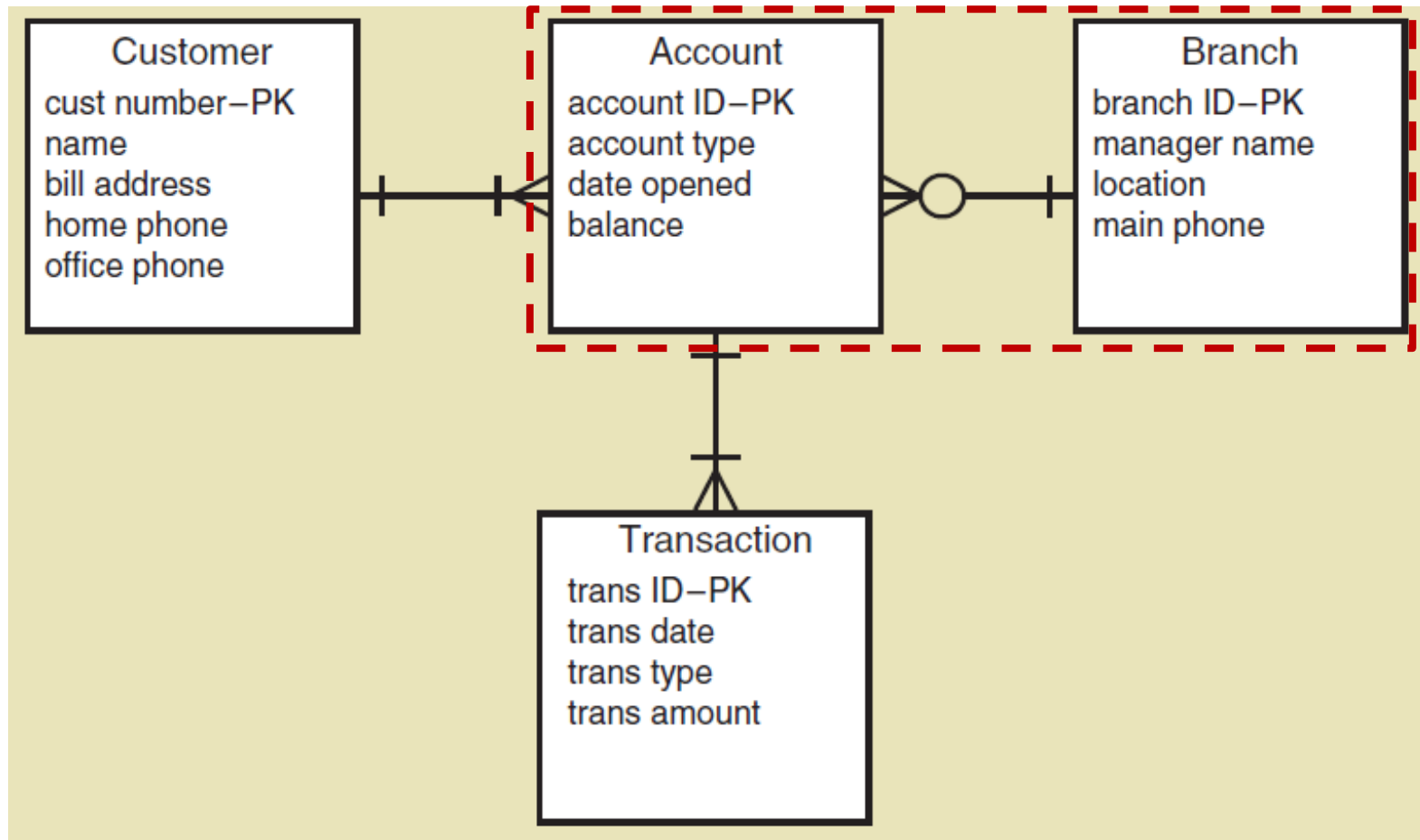


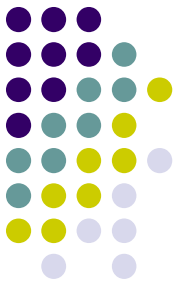
# An ERD for a Bank



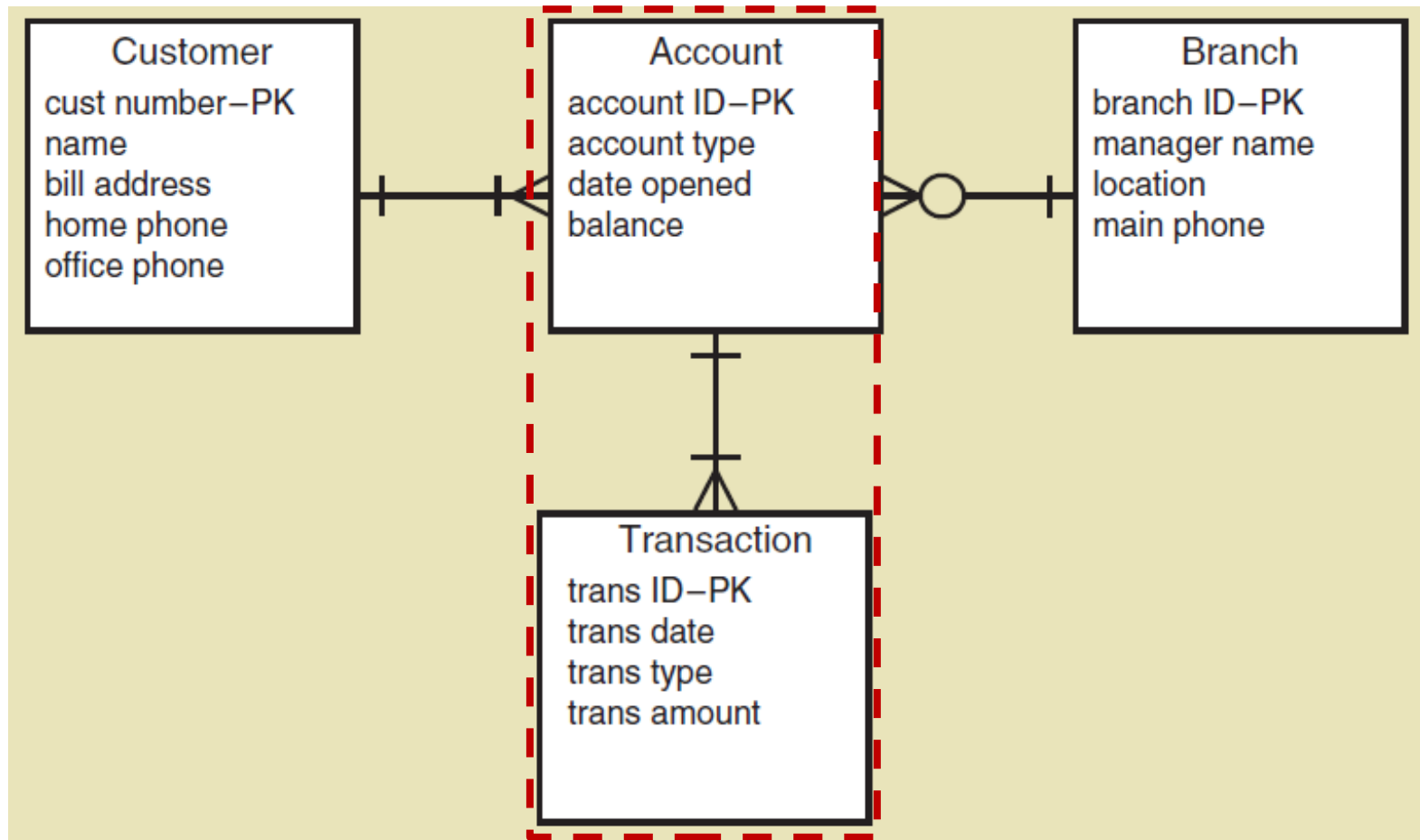


# An ERD for a Bank



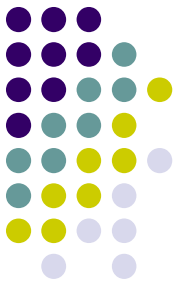


# An ERD for a Bank

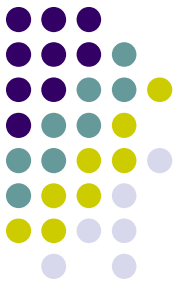


# Entity-Relationship Diagrams

## ERD



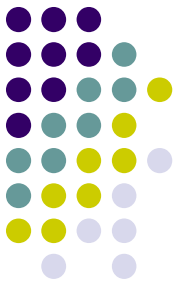
- It is not a UML diagram, but it is widely used by data analysts in database management （不是 UML 图，但在数据库管理中被广泛使用）
- There really is no standard notation, but most developers use the entity and crows feet notation shown in this text
- An ERD is not good for showing generalization/specialization relationships and whole part relationships （ ERD 不适合显示泛化 / 特化的关系和整体 - 部分的关系）



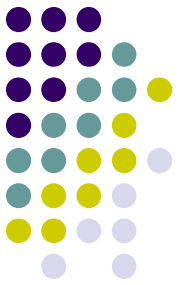
# Chapter 4 Outline

- “Things” in the Problem Domain
  - Data entities
  - Domain classes
- The Entity-Relationship Diagram
- The Domain Model Class Diagram (域模型类图)
- The Domain Model Class Diagram in the RMO example

# The Domain Model Class Diagram

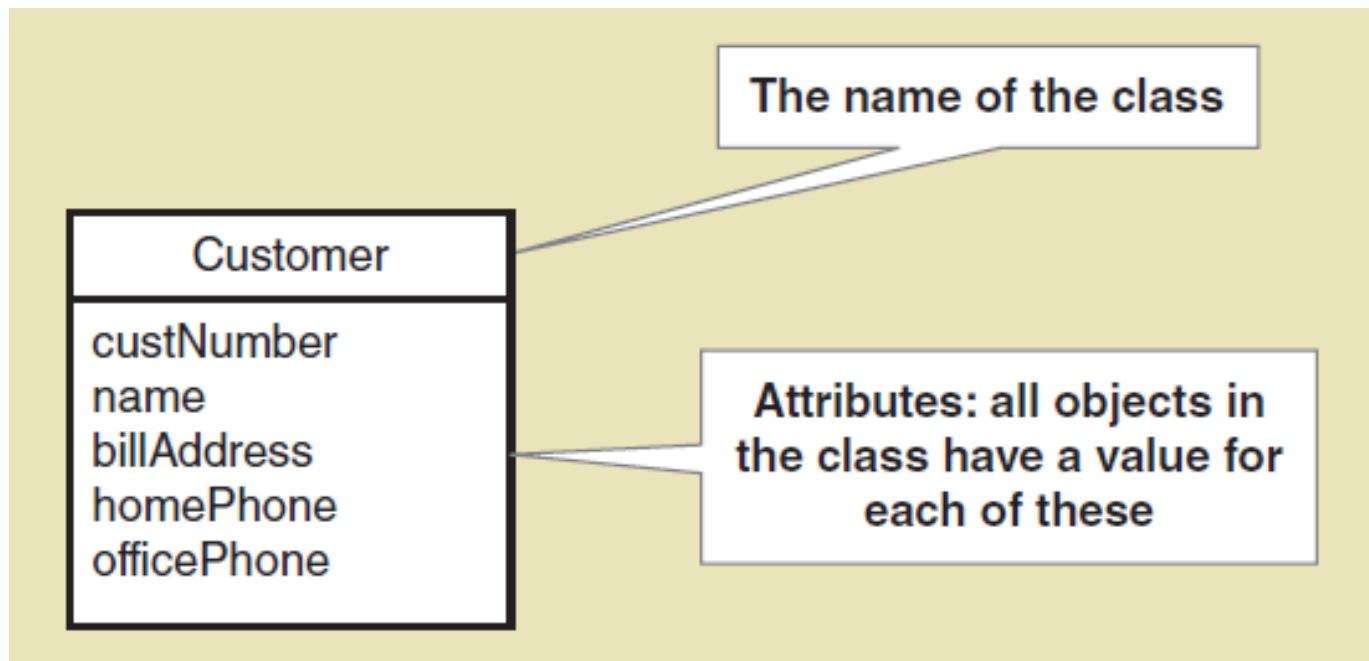


- Class (类)
  - A category of classification used to describe a collection of objects (用于描述一组对象的分类范畴)
- Domain Class (域类)
  - Classes that describe objects in the problem domain (问题域中的类)
- Class Diagram (类图)
  - A UML diagram that shows classes with attributes and associations (plus methods if it models software classes)
- Domain Model Class Diagram (域模型类图)
  - A class diagram that only includes classes from the problem domain, not software classes so no methods



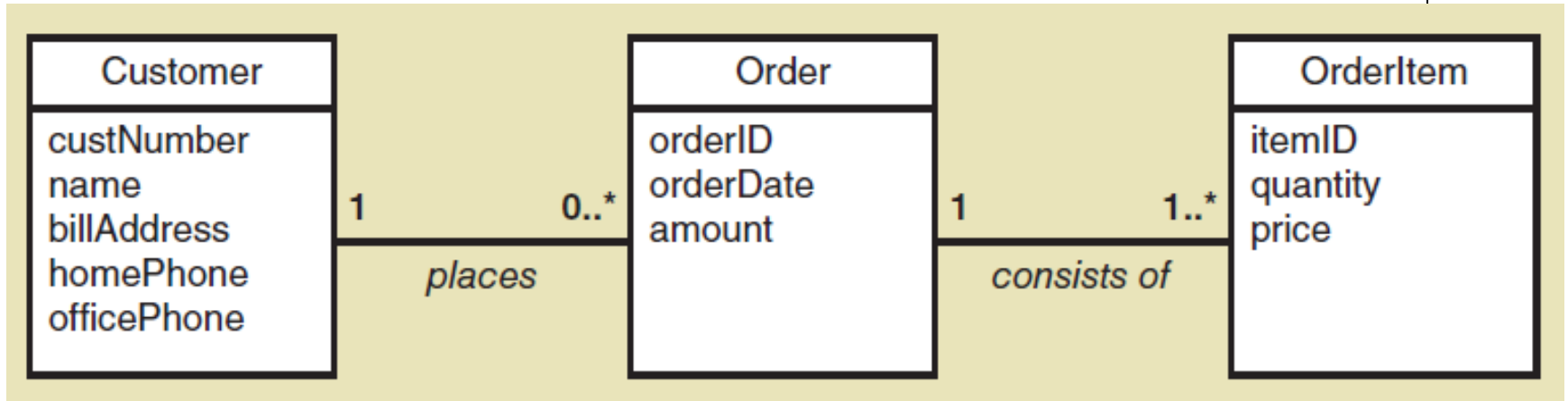
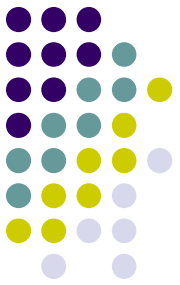
# Domain Class Notation

- Class name is always capitalized
- Attribute names are not capitalized and use camelback notation (words run together and second word is capitalized)

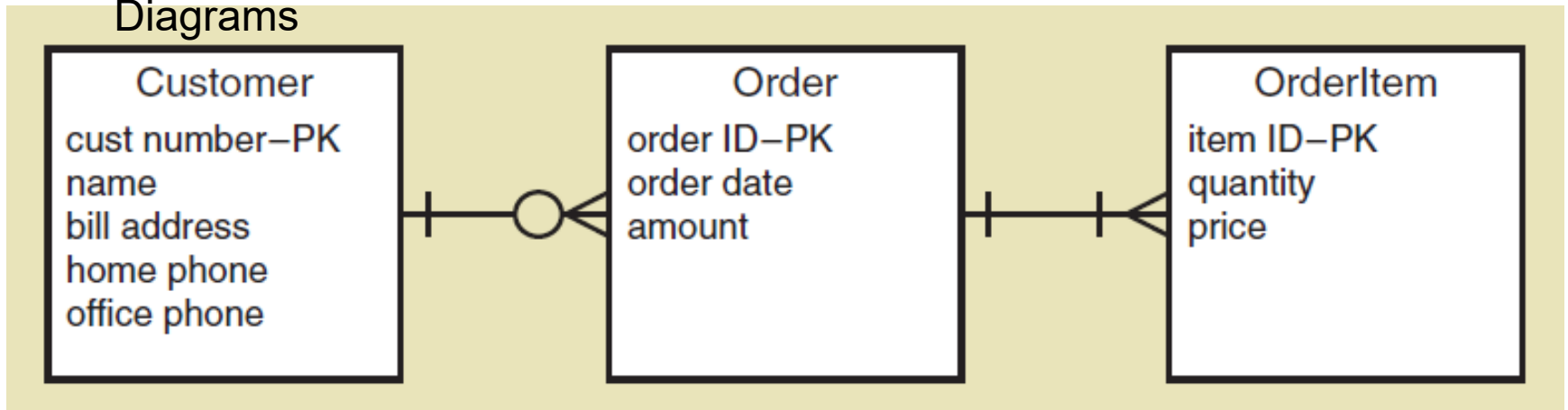




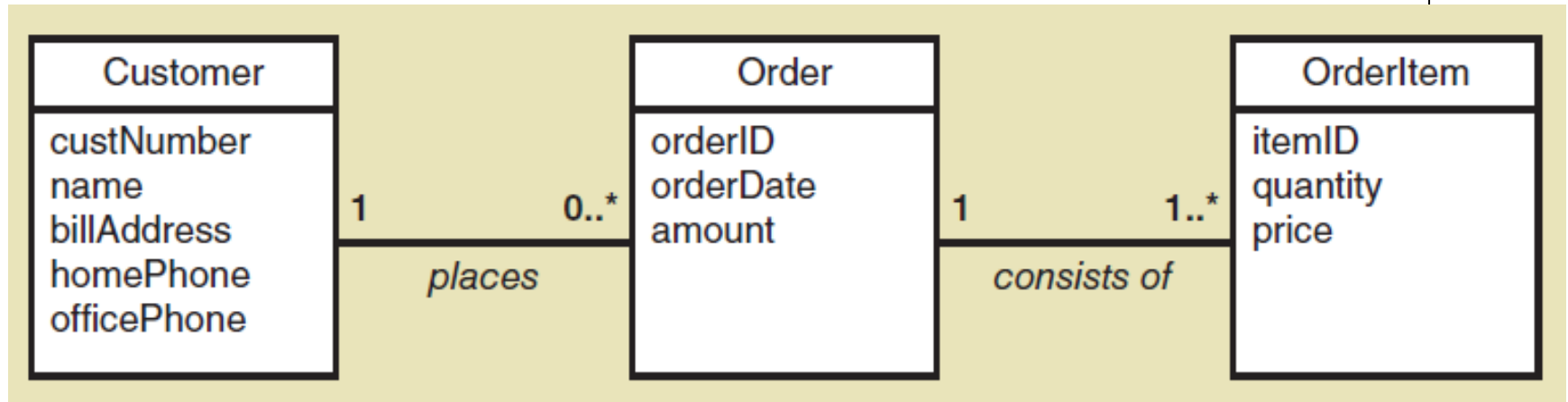
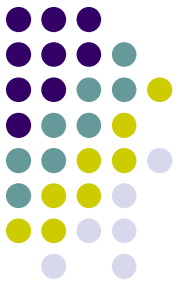
# A Simple Domain Model Class Diagram



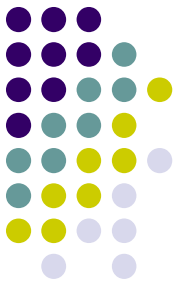
上： Domain Model Class Diagram; 下： Entity-Relationship Diagrams



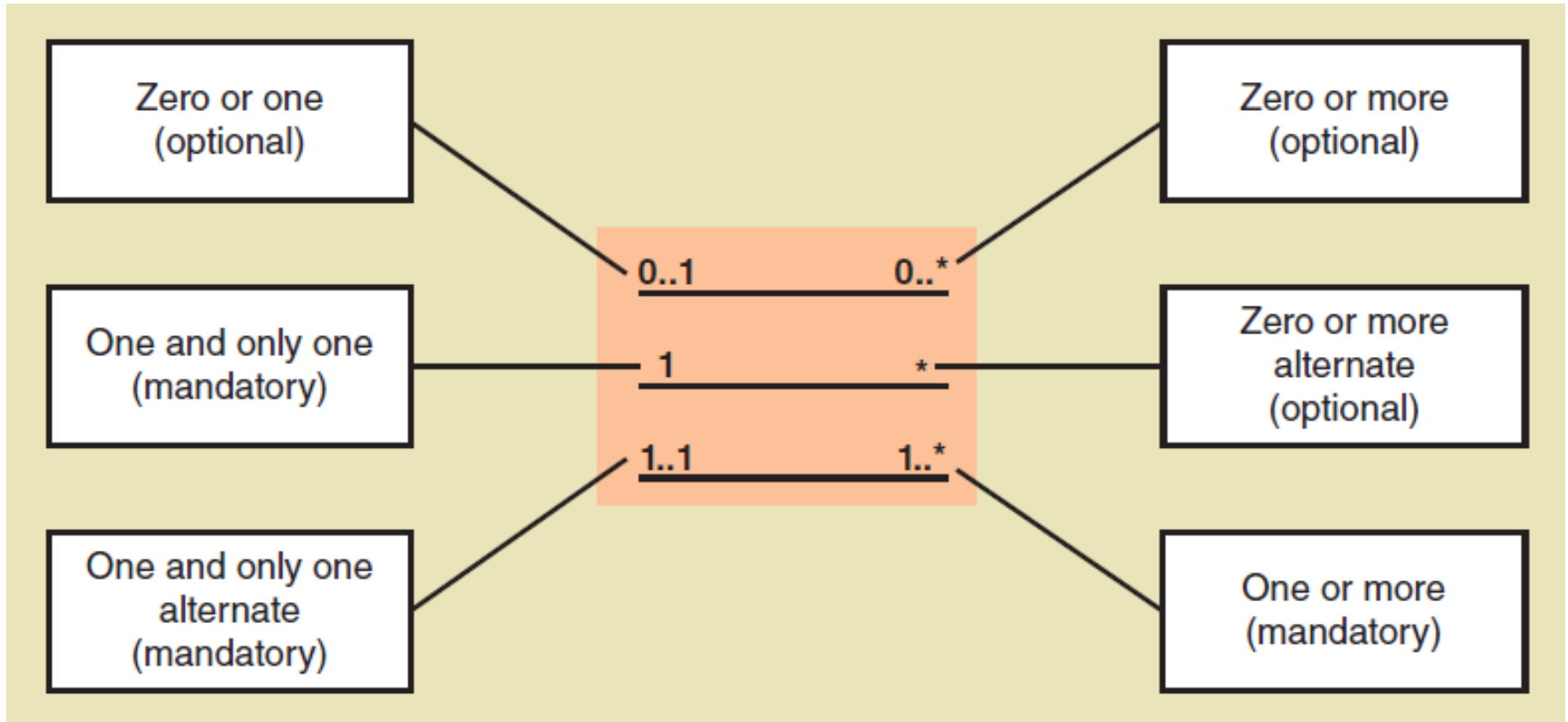
# A Simple Domain Model Class Diagram



- Note: This diagram matches the semantic net shown previously
  - A customer places zero or more orders
  - An order is placed by exactly one customer
  - An order consists of one or more order items
  - An order item is part of exactly one order

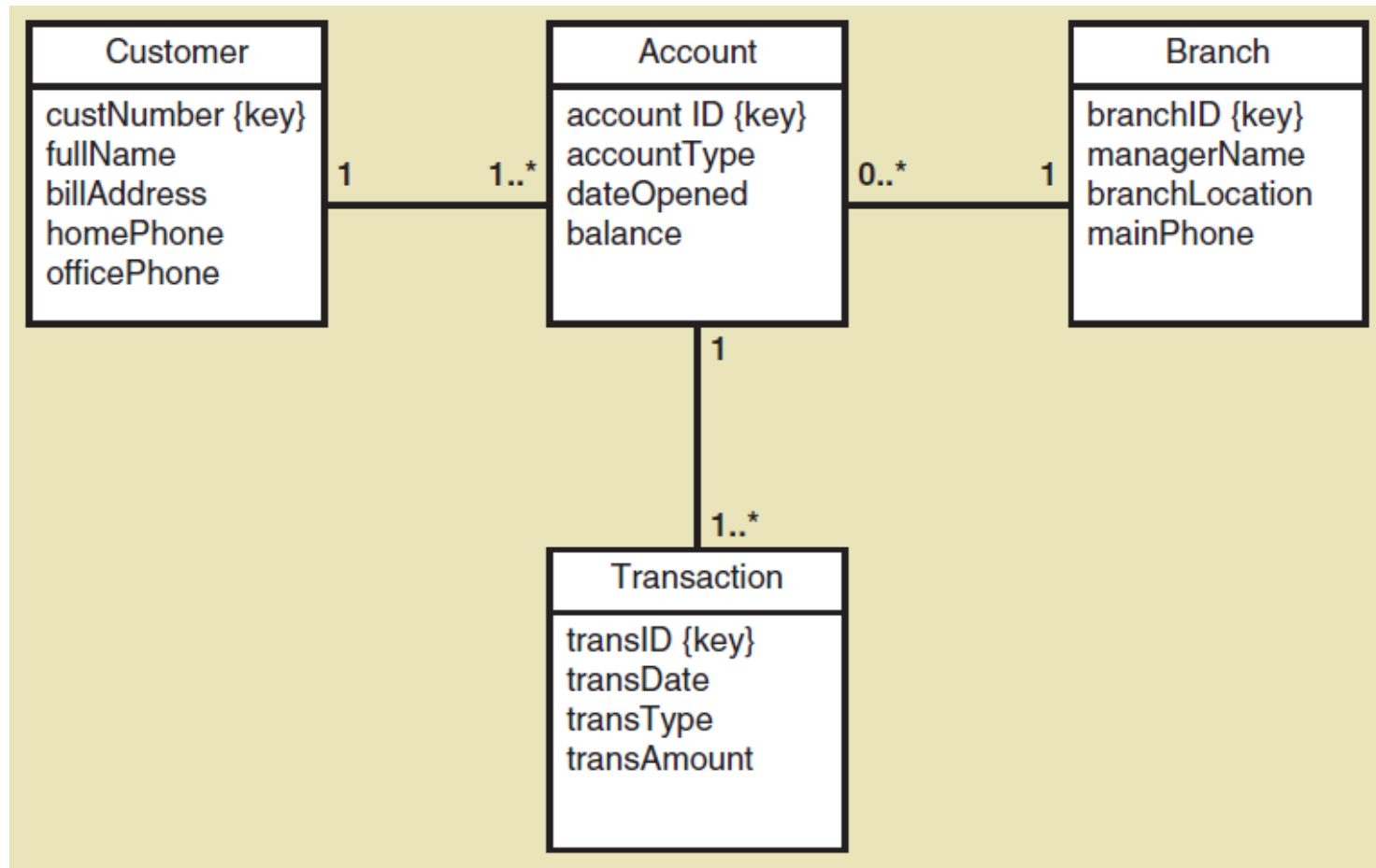
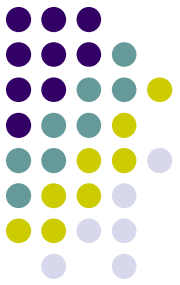


# UML Notation for Multiplicity

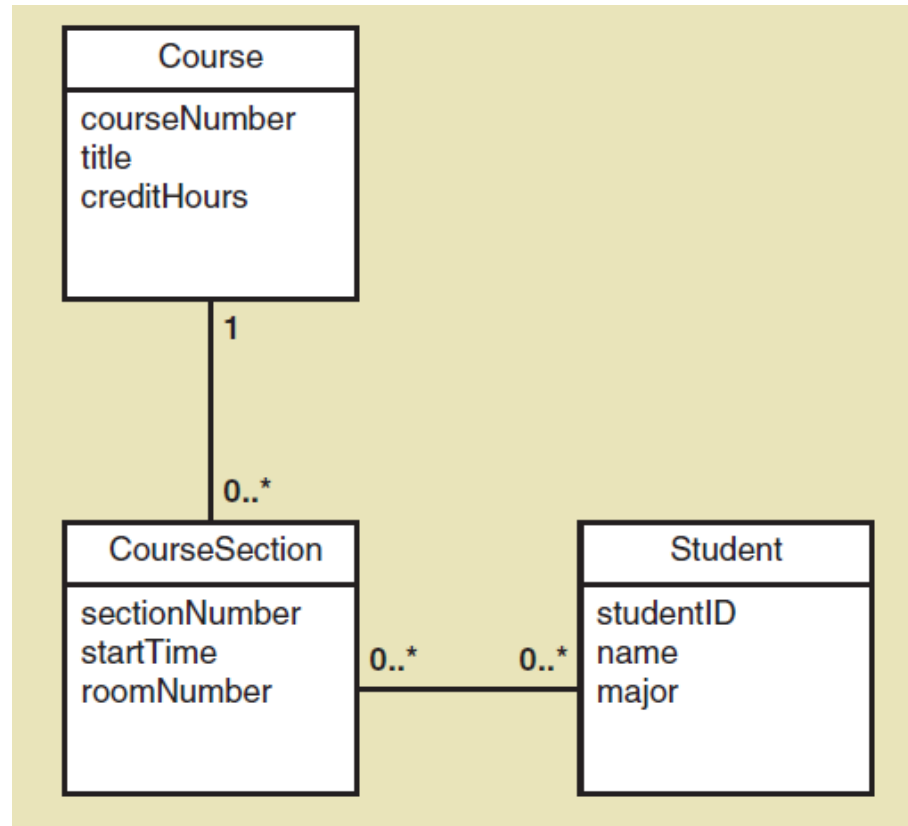
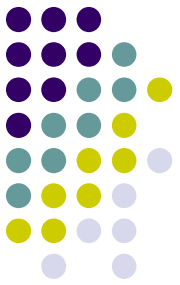


只要可以为 0 ， 就是  
optional

# Domain Model Class Diagram for a bank with many branches



# Domain Model Class Diagram for course enrollment at a university

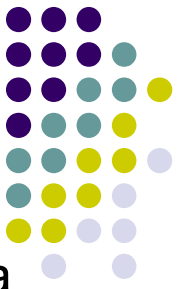


- Where is each student's grade remembered in this model? (在这个模型中，每个学生的成绩记在哪里？)
  - Each section has many grades and each grade is association with a student
  - Each student has many grades and each grade is association with a section

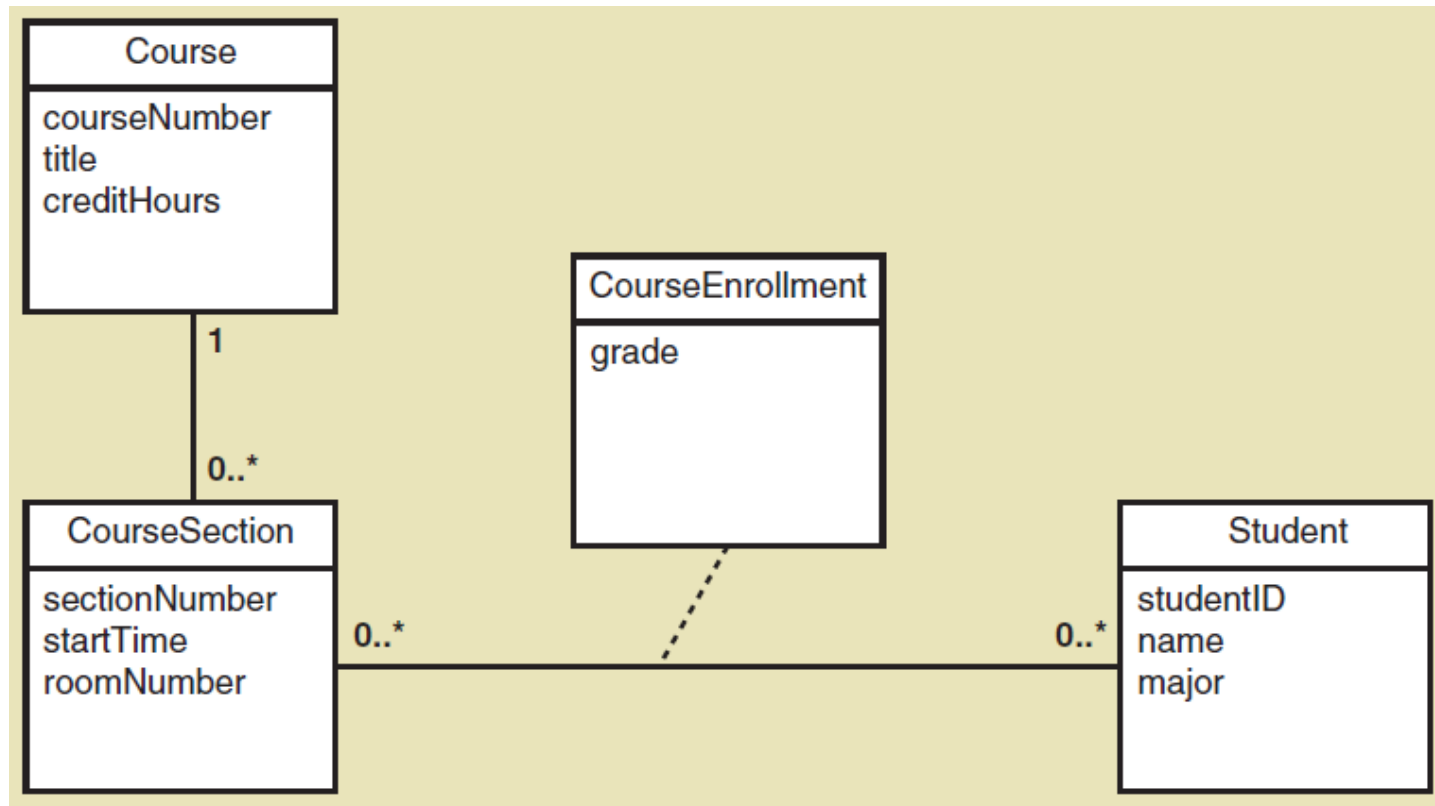
Introduction to Systems Analysis and Design, 6th Edition

45

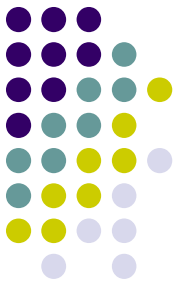
# Refined Course Enrollment Model with an Association Class CourseEnrollment



- Association class (关联类) — an association that is treated as a class in a many to many association because it has attributes that need to be remembered, such as grade



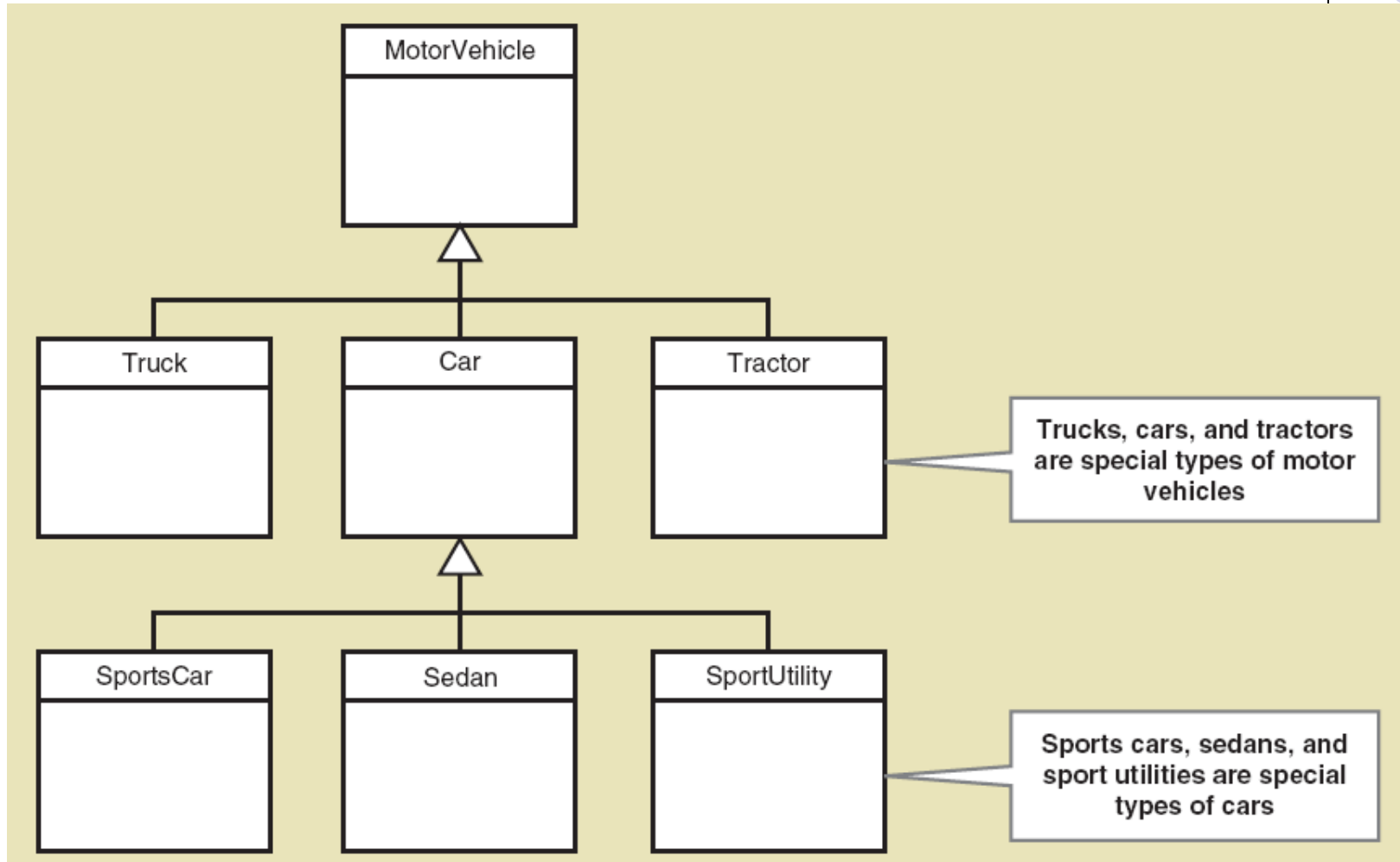
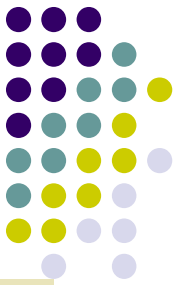
# More Complex Issues about Classes: Generalization/Specialization Relationships



- Generalization/Specialization (泛化 / 特化)
  - A hierarchical relationship where subordinate classes are special types of the superior classes. Often called an Inheritance Hierarchy
- Superclass (超类)
  - the superior or more general class in a generalization/specialization hierarchy (泛化 / 特化中更通用的类)
- Subclass (子类)
  - the subordinate or more specialized class in a generalization/specialization hierarchy (泛化 / 特化中从属的类)
- Inheritance (继承)
  - the concept that subclasses classes inherit characteristics of the more general superclass

# Generalization/Specialization

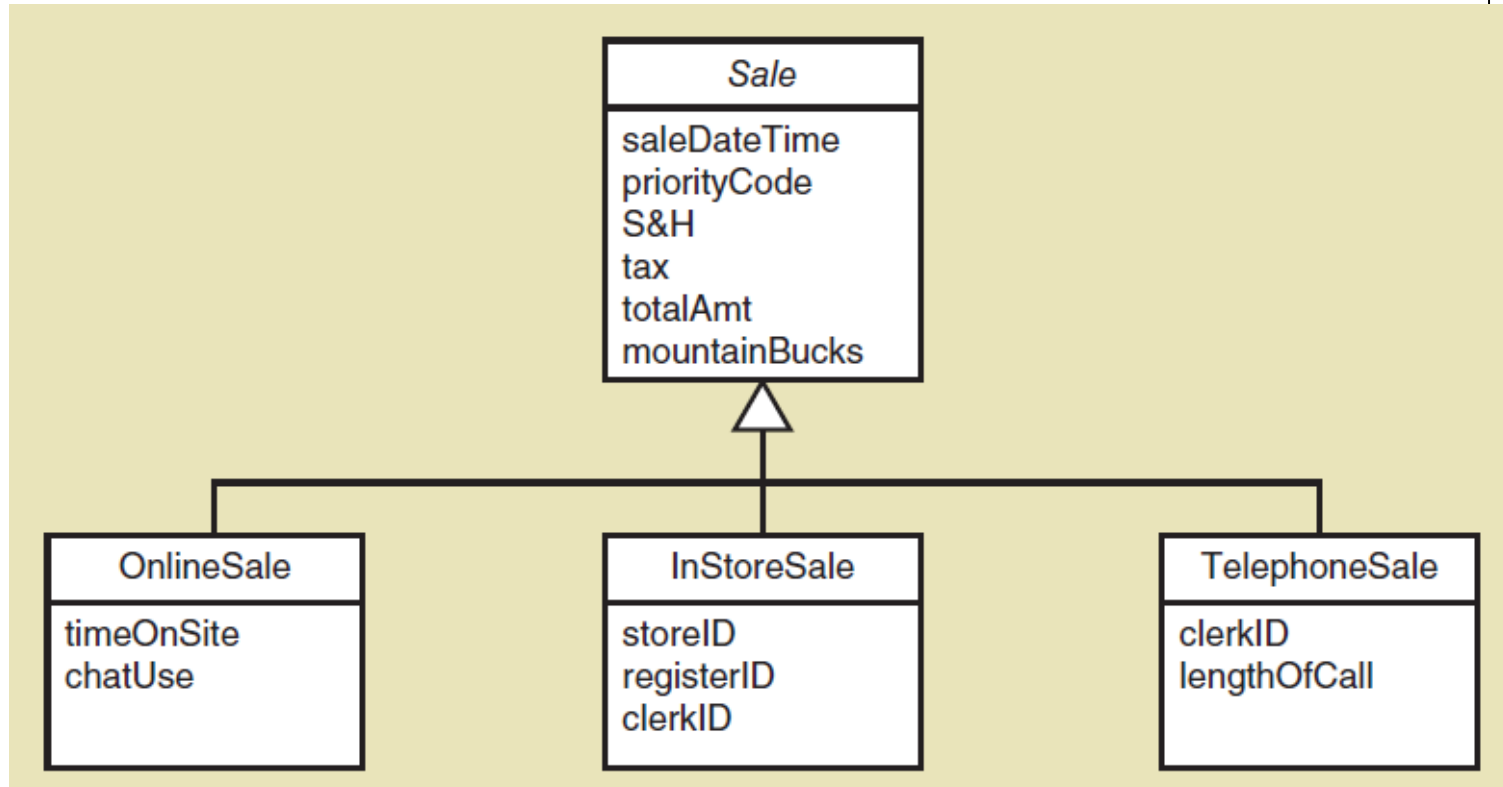
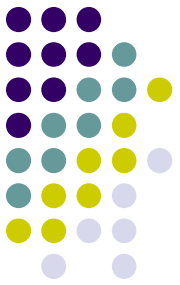
## Inheritance





# Generalization/Specialization

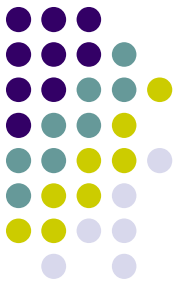
## Inheritance for RMO Three Types of Sales



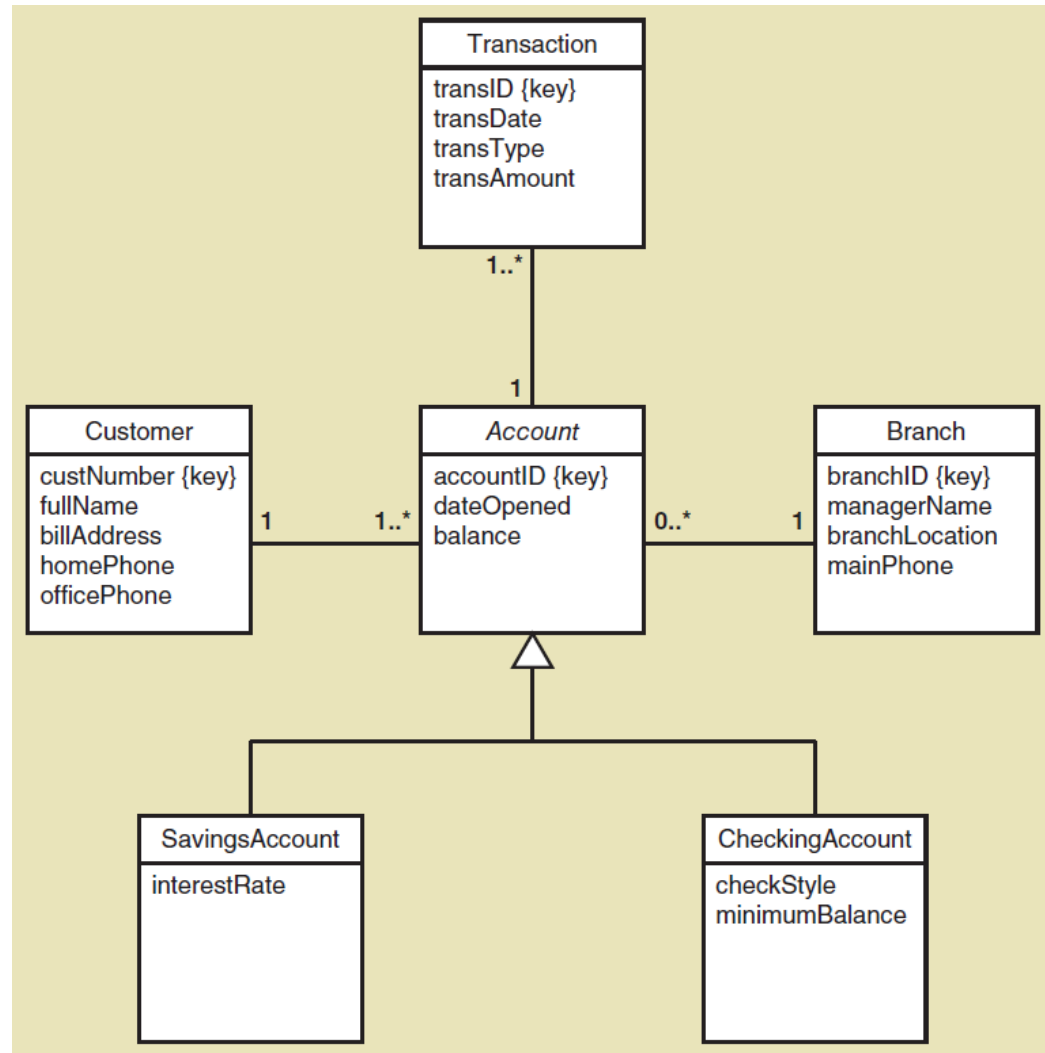
- **Abstract class** (抽象类) — a class that allow subclasses to inherit characteristics but **never gets instantiated** (永远不会被实例化) . In Italics (*Sale* above)
- **Concrete class** (具体类) — a class that can have instances (可以有实例的类)

# Generalization/Specialization

## Inheritance for the Bank with Special Types of Accounts

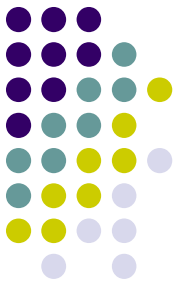


- A SavingsAccount (储蓄账户) has 4 attributes
- A CheckingAccount (支票账户) Has 5 attributes
- Note: the subclasses inherit the associations, too



# More Complex Issues about Classes:

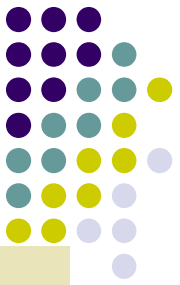
## Whole Part Relationships



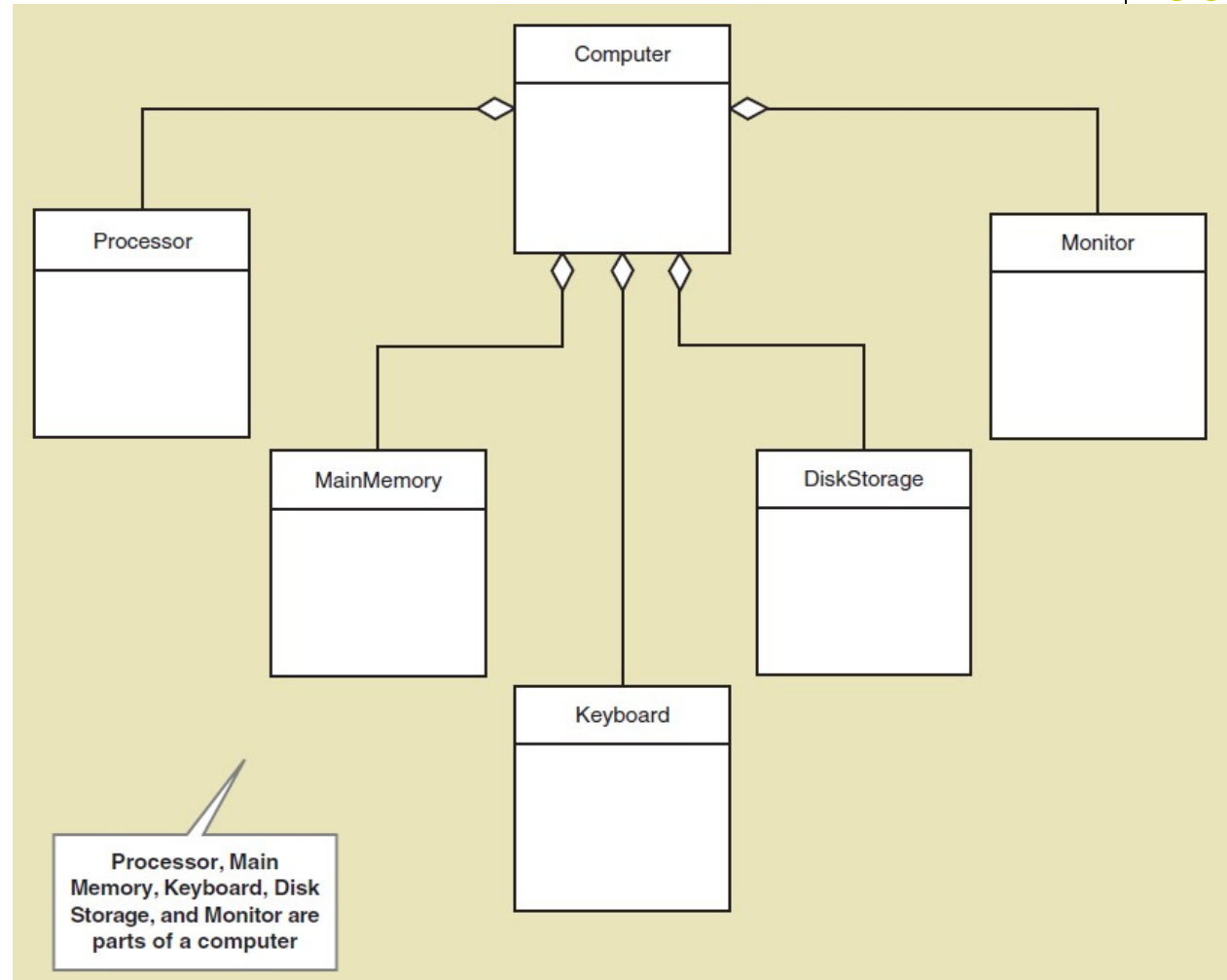
- Whole-part relationship (整体 - 部分关联) — a relationship between classes where one class is part of or a component portion of another class (其中一个类是另一个类的一部分或组成部分)
- Aggregation (聚合) — a whole part relationship where the component part exists separately and can be removed and replaced (UML diamond symbol, next slide)
  - Computer has disk storage devices (计算机有磁盘存储)
  - Car has wheels (汽车有轮子)
- Composition (合成) — a whole part relationship where the parts can no longer be removed (filled in diamond symbol)
  - Hand has fingers (手有手指)
  - Chip has circuits (芯片有电路)

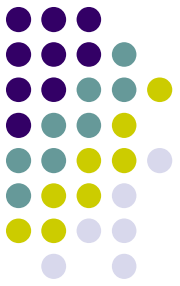
# Whole Part Relationships

## Computer and its Parts



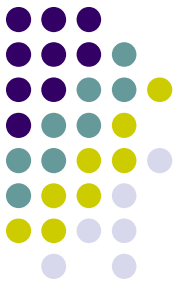
- Note: this is aggregation, with diamond symbol.
- Whole part can have multiplicity symbols, too (not shown)





# More on UML Relationships

- There are actually three types of ***relationships*** in class diagrams （在类图中有三种类型的关系）
  - Association Relationships （关联关系）
    - These are associations discussed previously, just like ERD relationships
  - Whole Part Relationships （整体 - 部分关系）
    - One class is a component or part of another class
  - Generalizations/Specialization Relationships （泛化 / 特化关系）
    - Inheritance

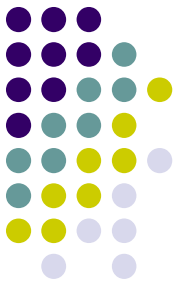


# Chapter 4 Outline

- “Things” in the Problem Domain
  - Data entities
  - Domain classes
- The Entity-Relationship Diagram
- The Domain Model Class Diagram
- The Domain Model Class Diagram in the RMO example

# RMO CSMS Project

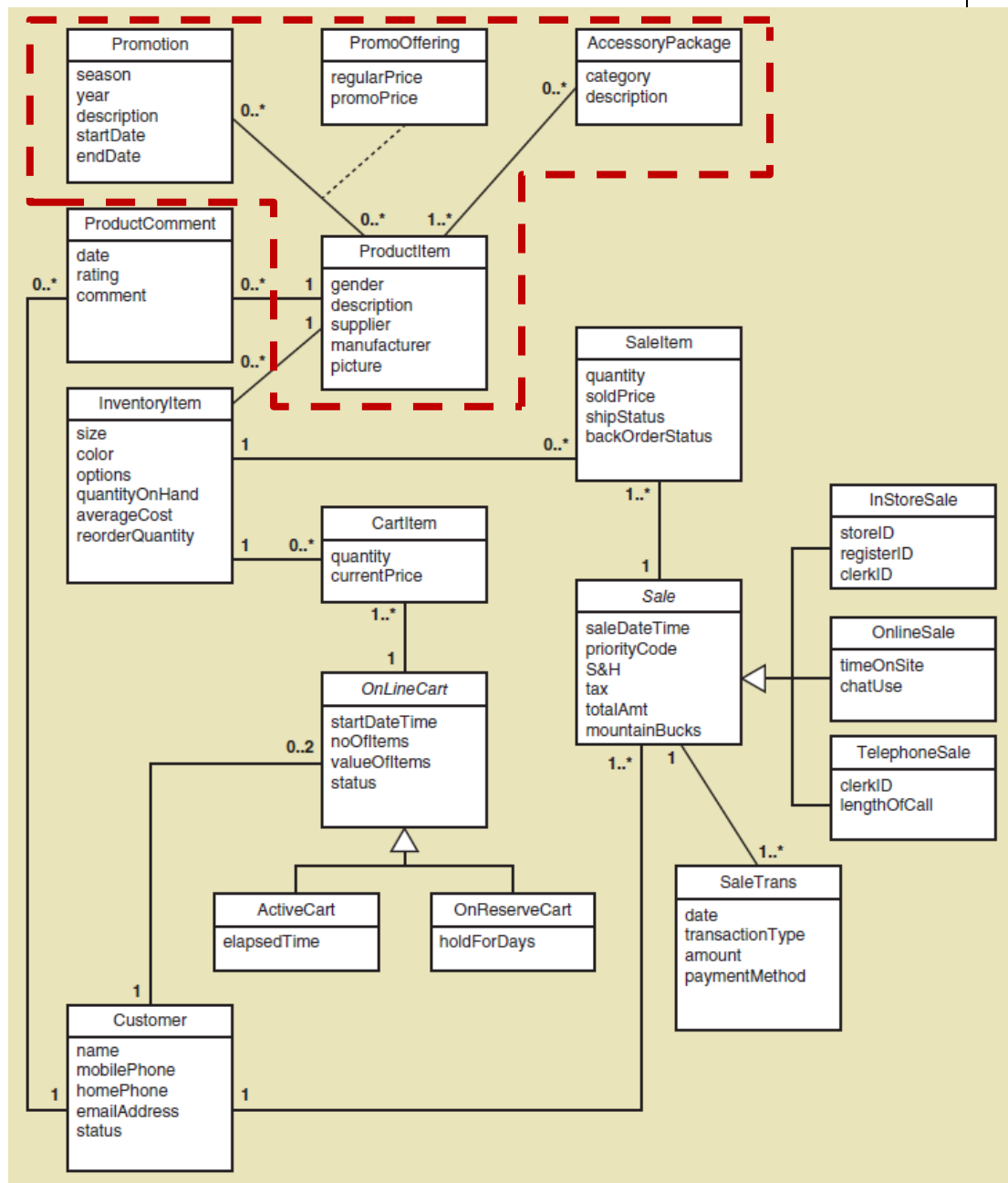
## Domain Model Class Diagrams



- There are several ways to create the domain model class diagram for a project
- RMO CSMS has 27 domain classes overall  
( RMO CSMS 总共有 27 个域类)
- Can create one domain model class diagram per subsystem for those working on a subsystem (可以为每个子系统创建一个域模型类图)
- Can create one overall domain model class diagram to provide an overview of the whole system (可以创建一个整体的域模型类图来提供整个系统的概览)

# RMO CSMS Project Sales Subsystem Domain Model Class Diagrams

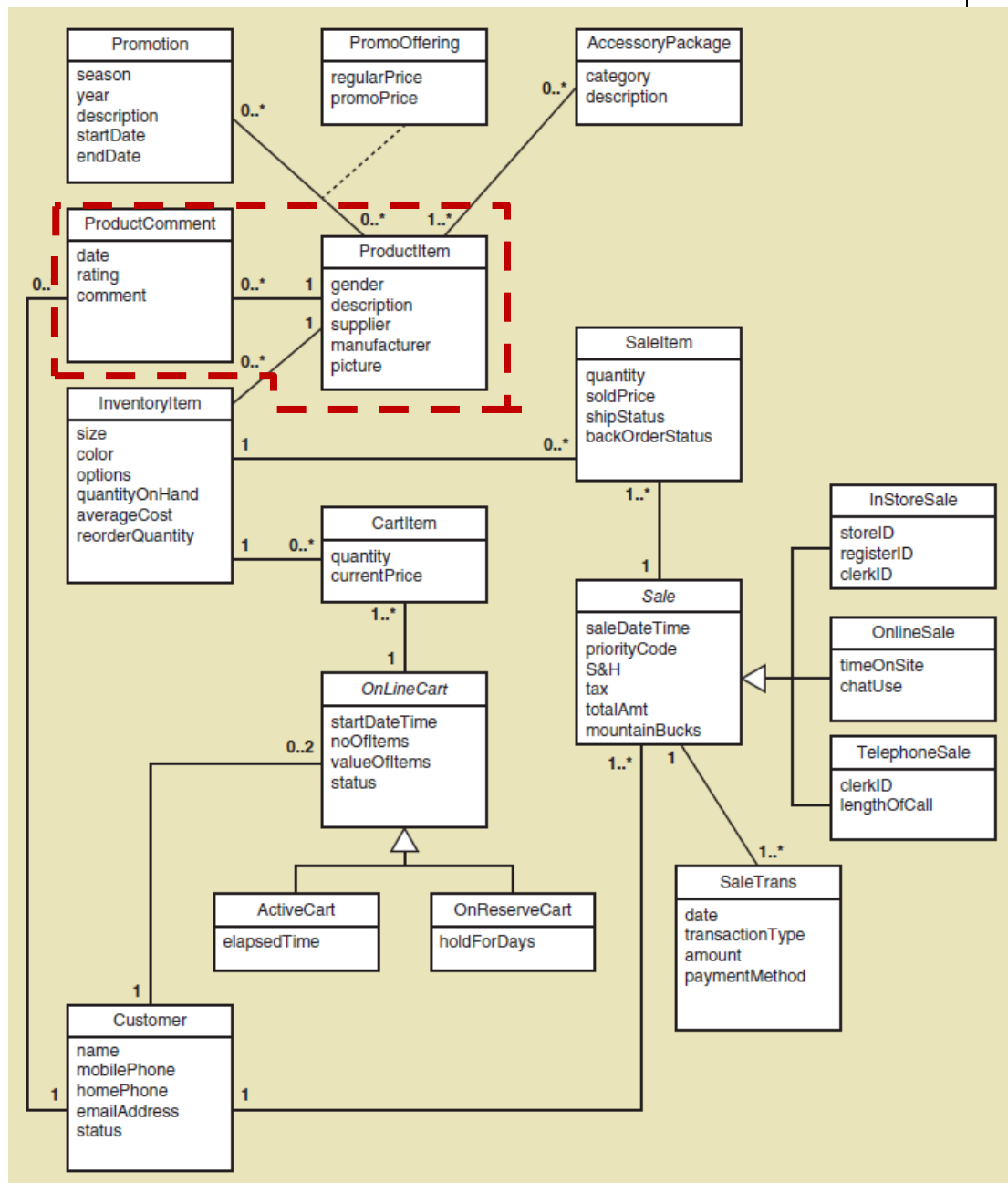
销售子系统  
域模型类图





# RMO CSMS Project

## Sales Subsystem Domain Model Class Diagrams



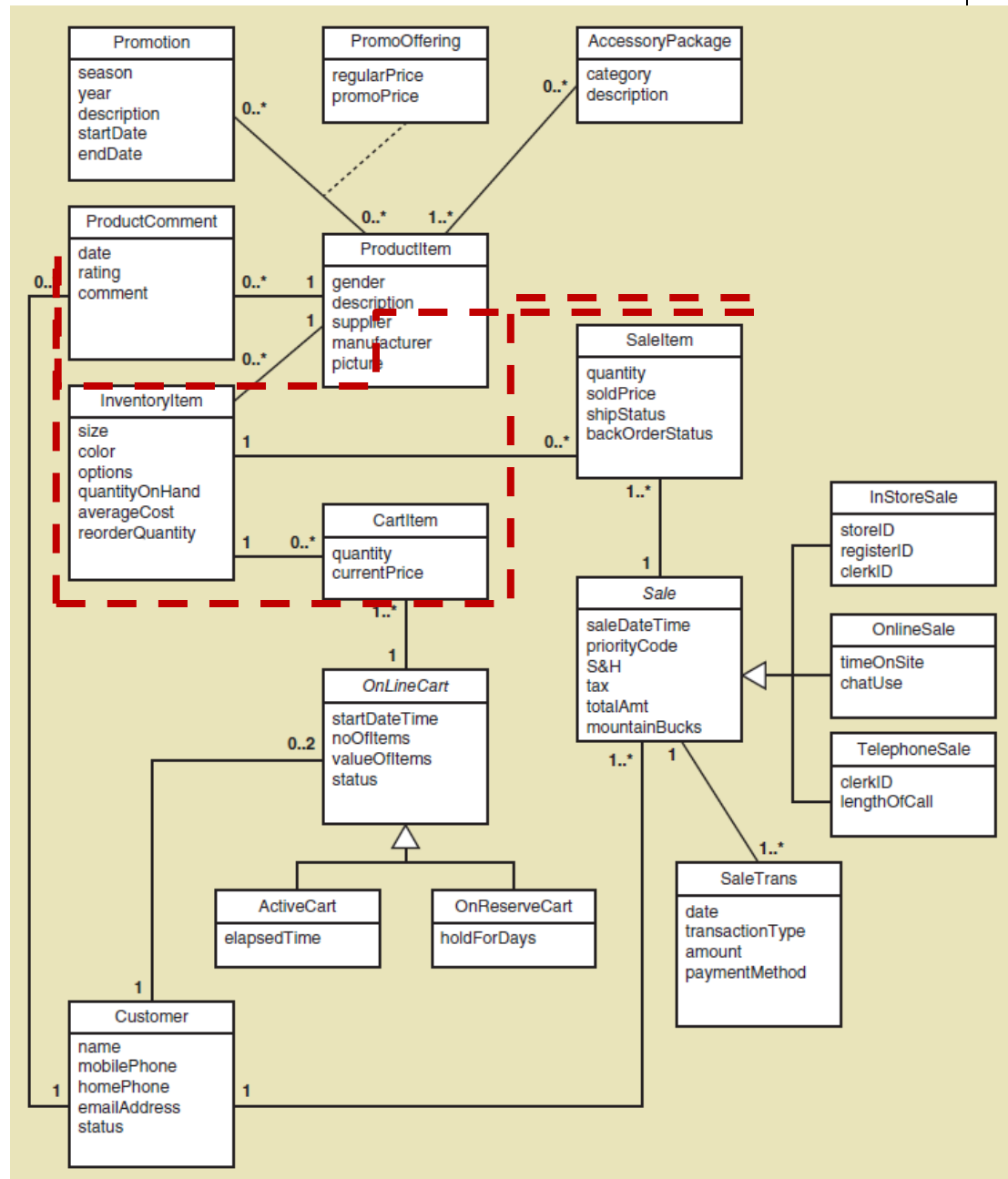
# RMO CSMS

## Project

### Sales Subsystem

### Domain Model

### Class Diagrams



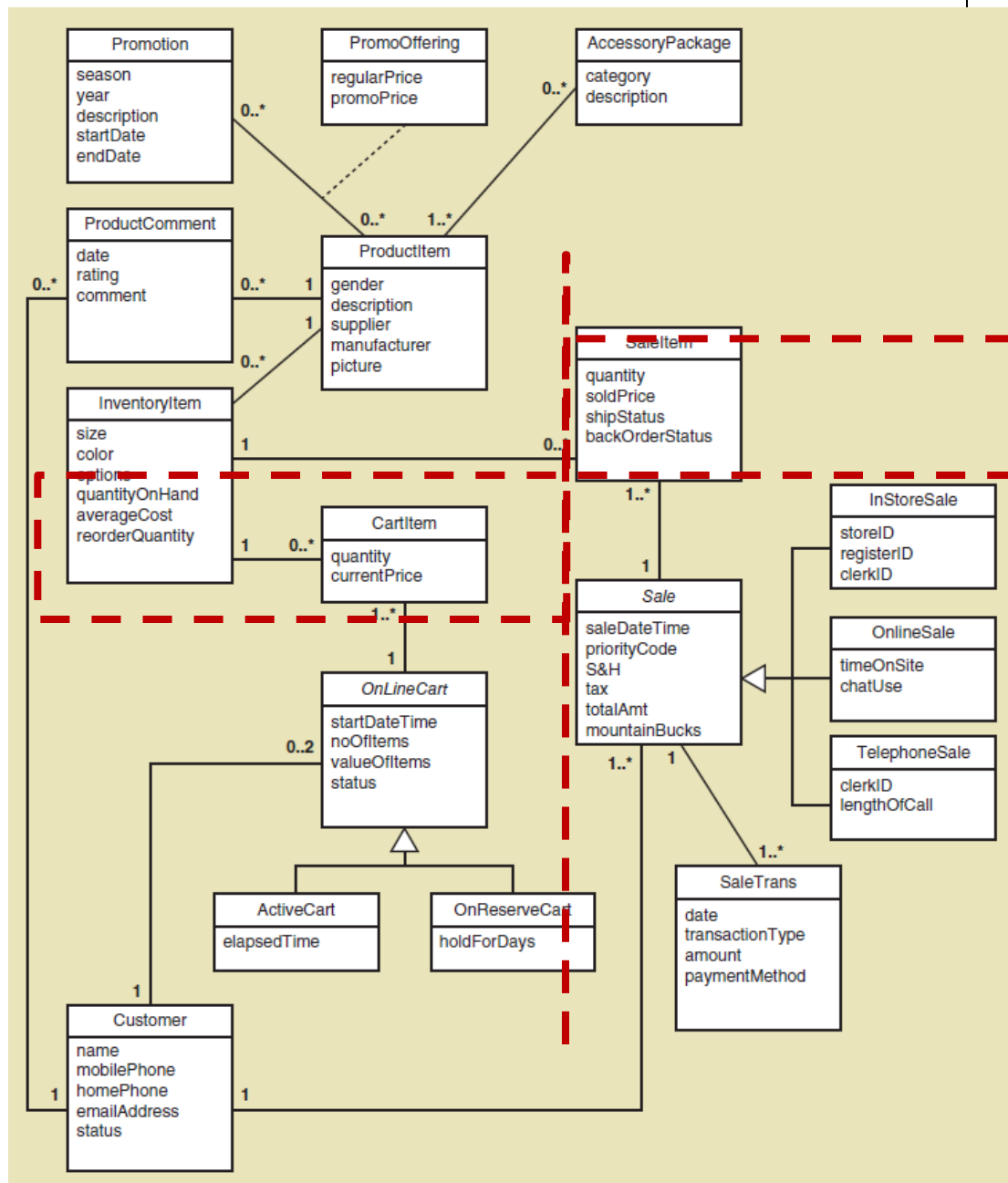
# RMO CSMS

## Project

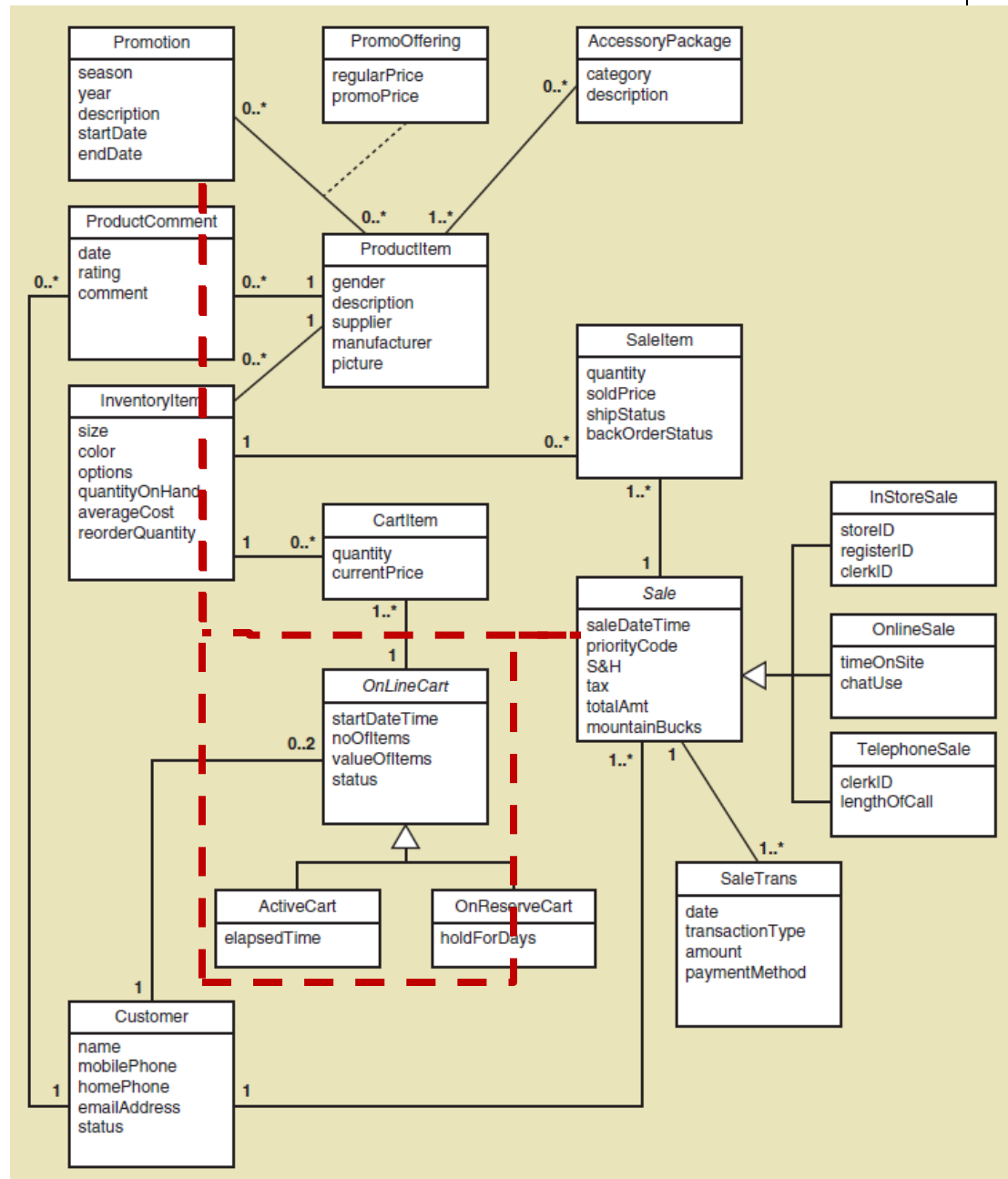
### Sales Subsystem

### Domain Model

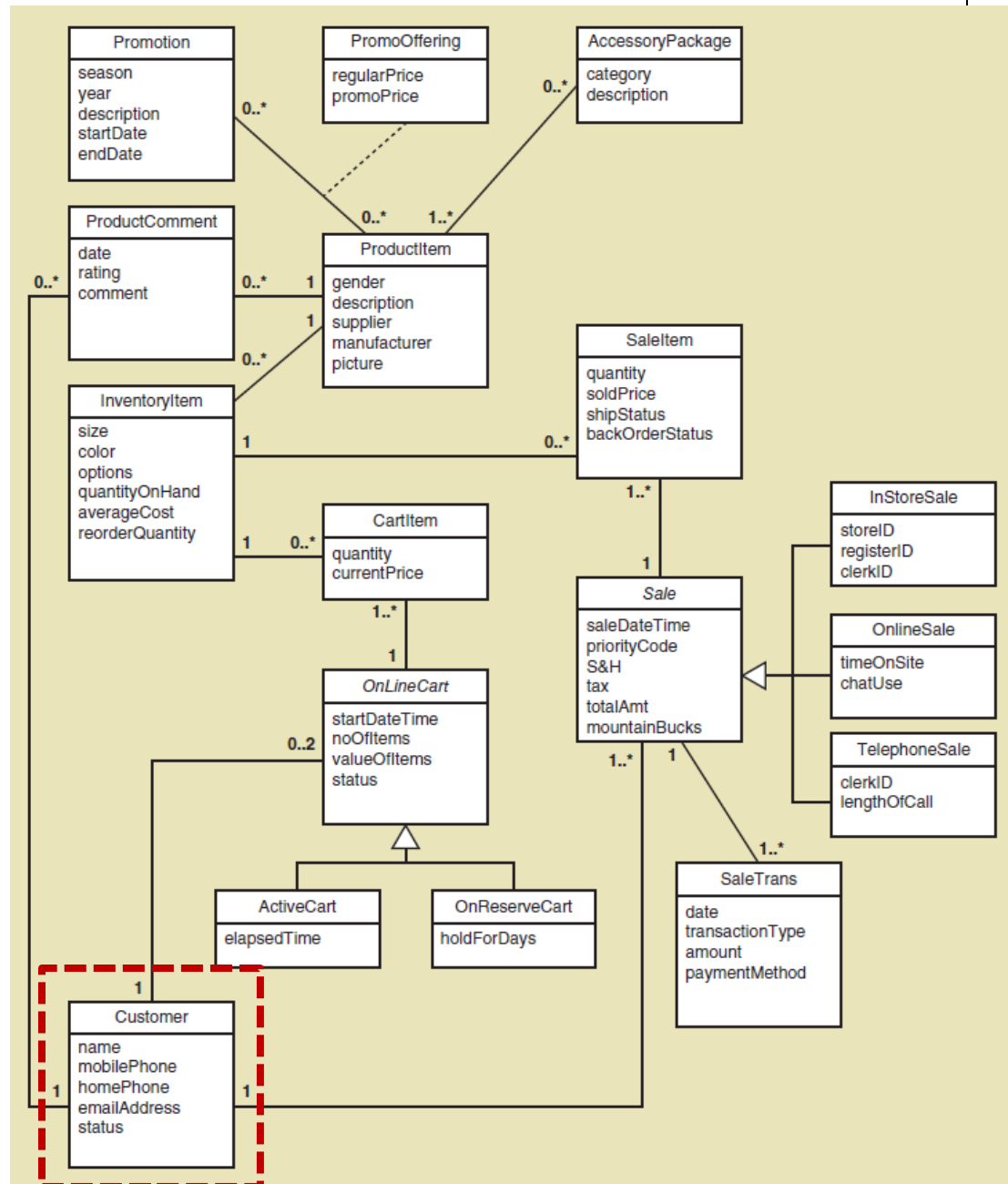
### Class Diagrams



# RMO CSMS Project Sales Subsystem Domain Model Class Diagrams

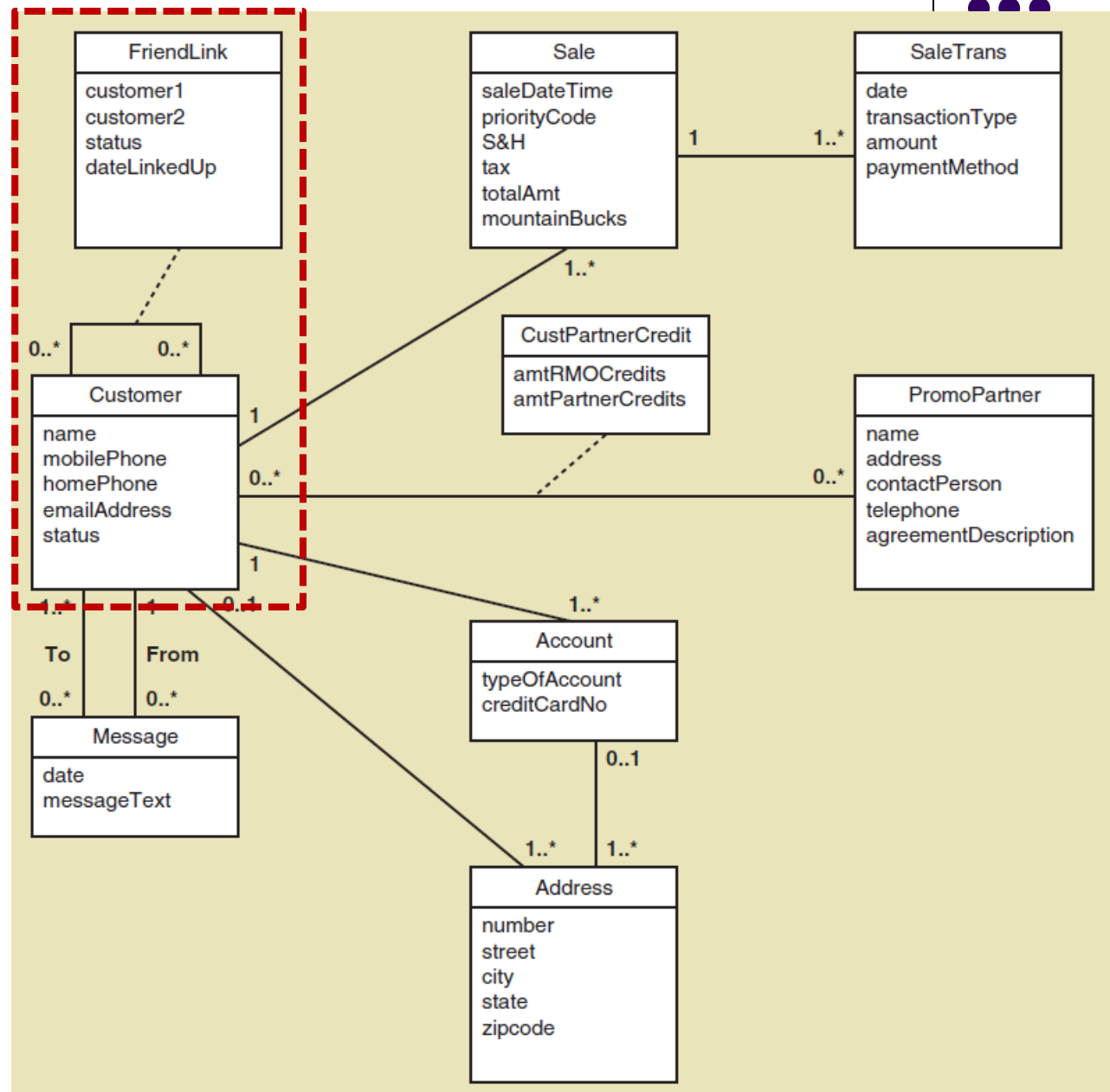


# RMO CSMS Project Sales Subsystem Domain Model Class Diagrams



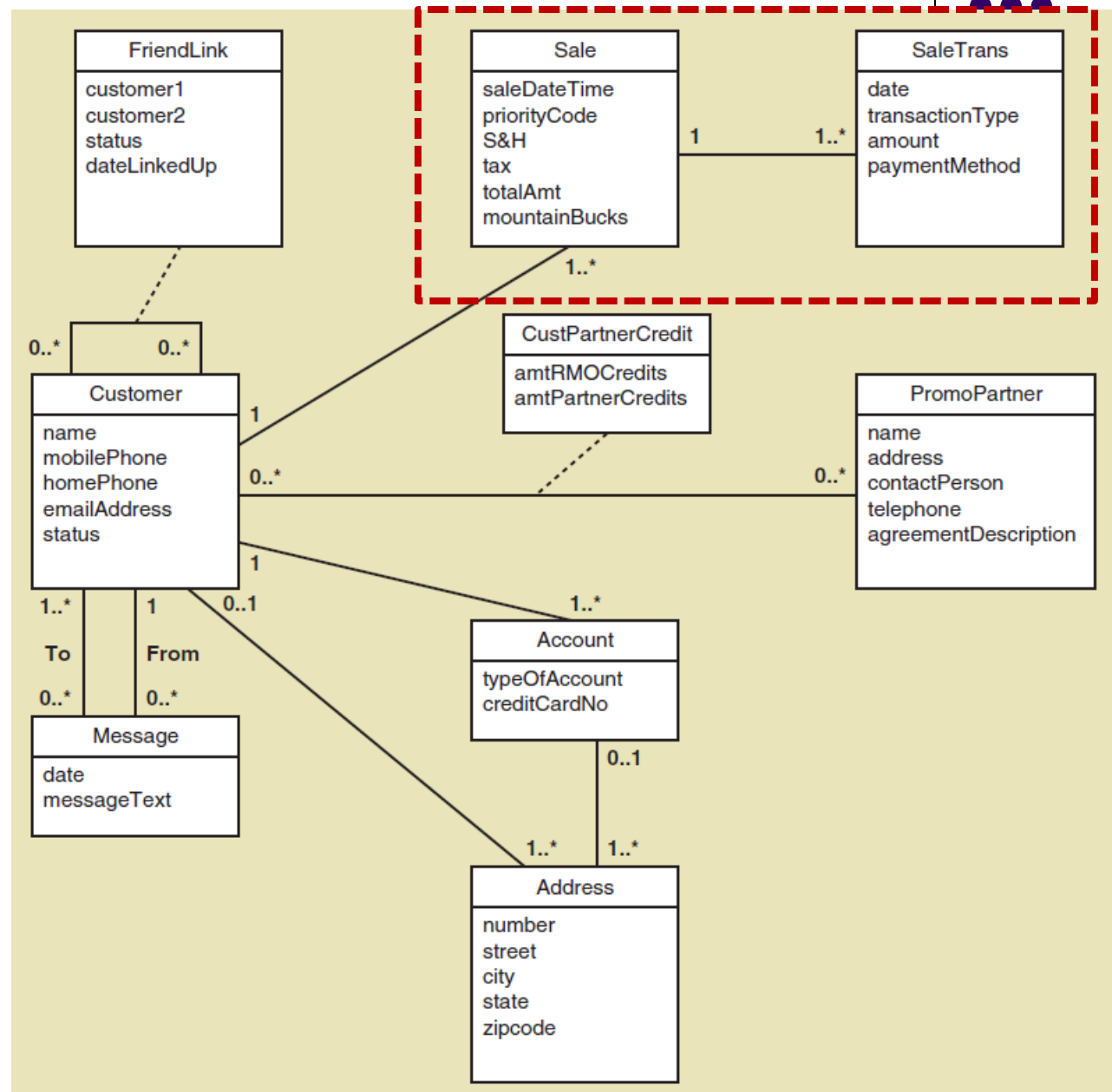
# RMO CSMS Project Customer Account Subsystem Domain Model Class Diagram

客户帐户子系统  
域模型类图

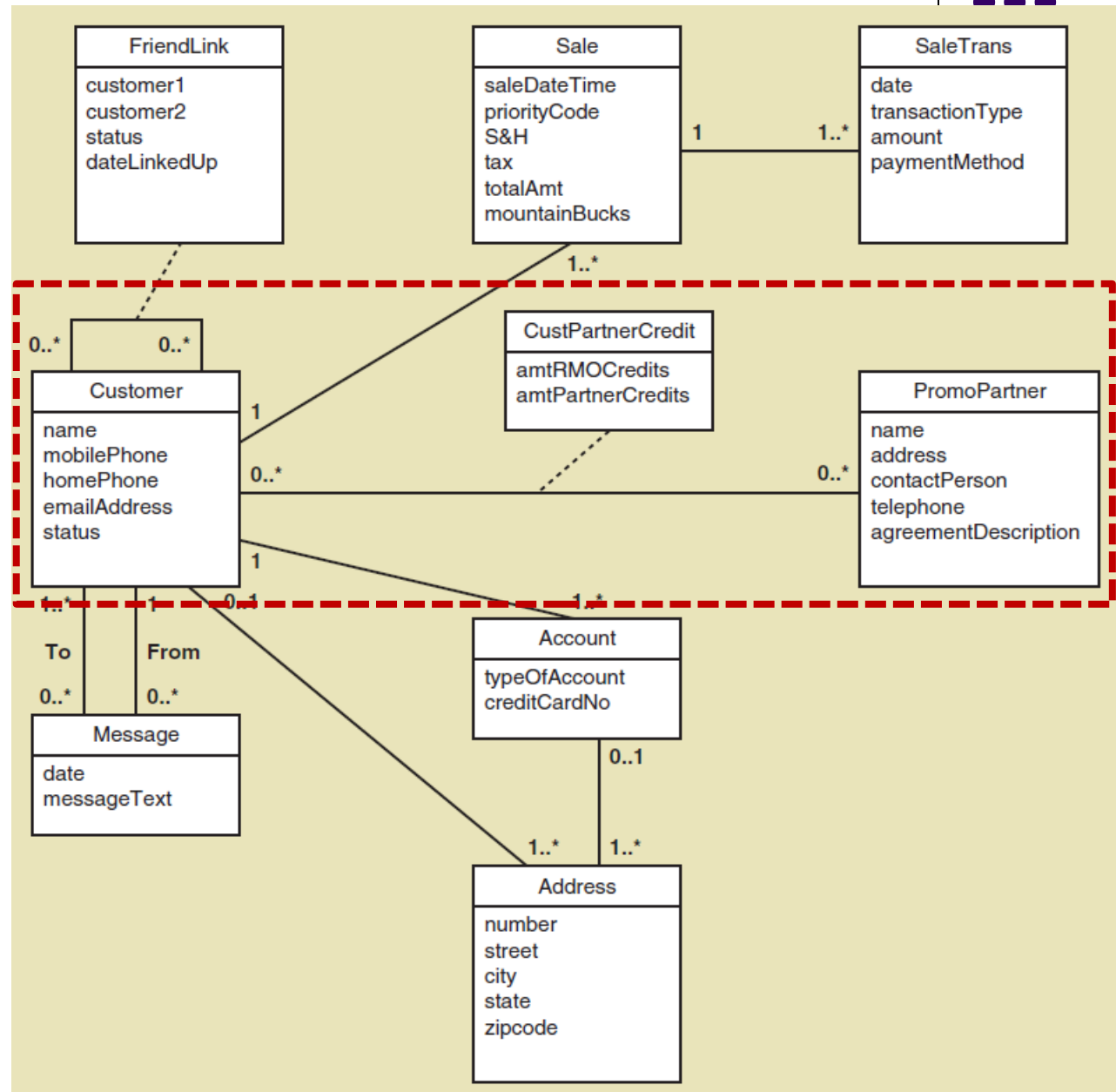


# RMO CSMS Project

## Customer Account Subsystem Domain Model Class Diagram

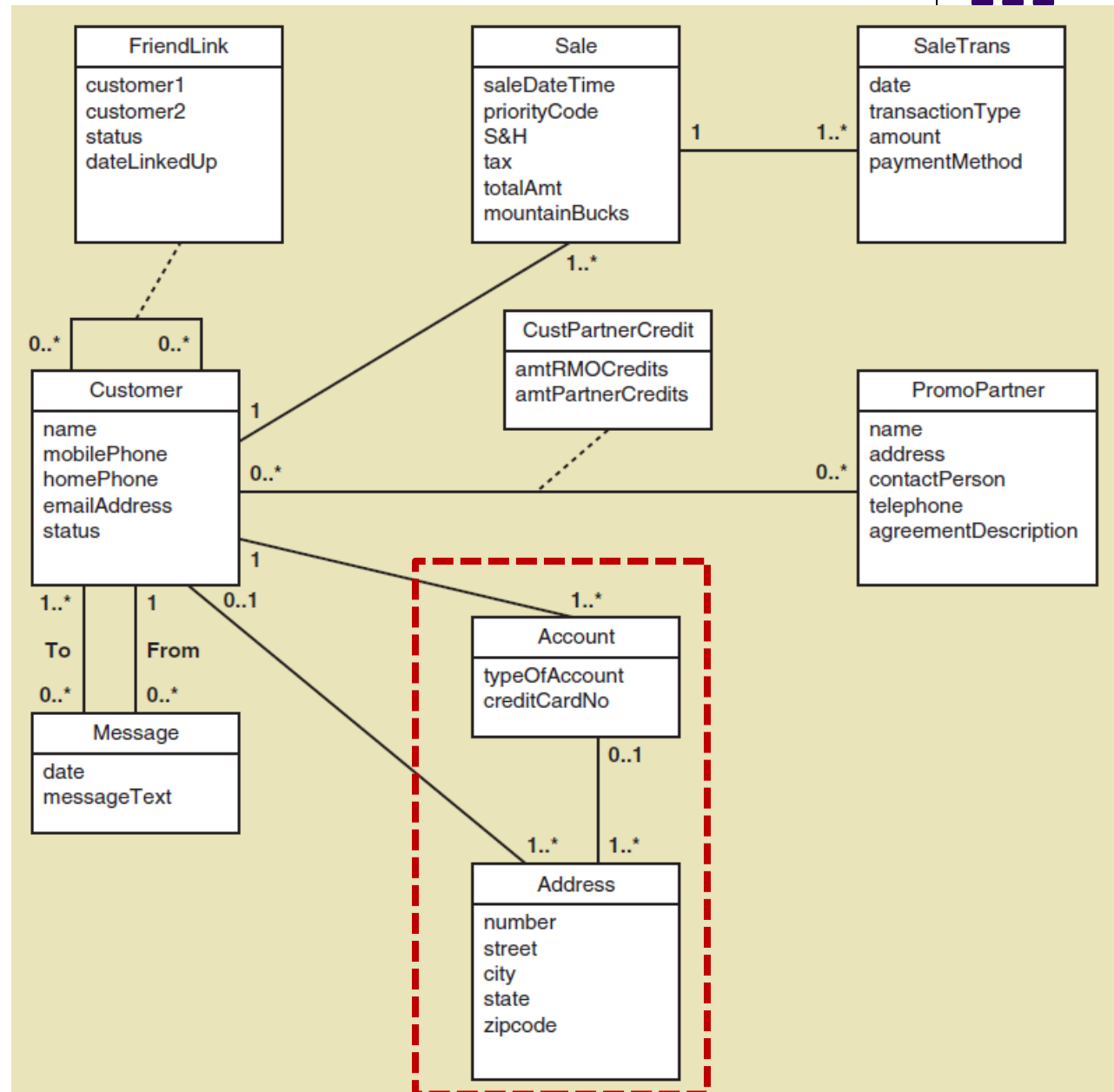


# RMO CSMS Project Customer Account Subsystem Domain Model Class Diagram

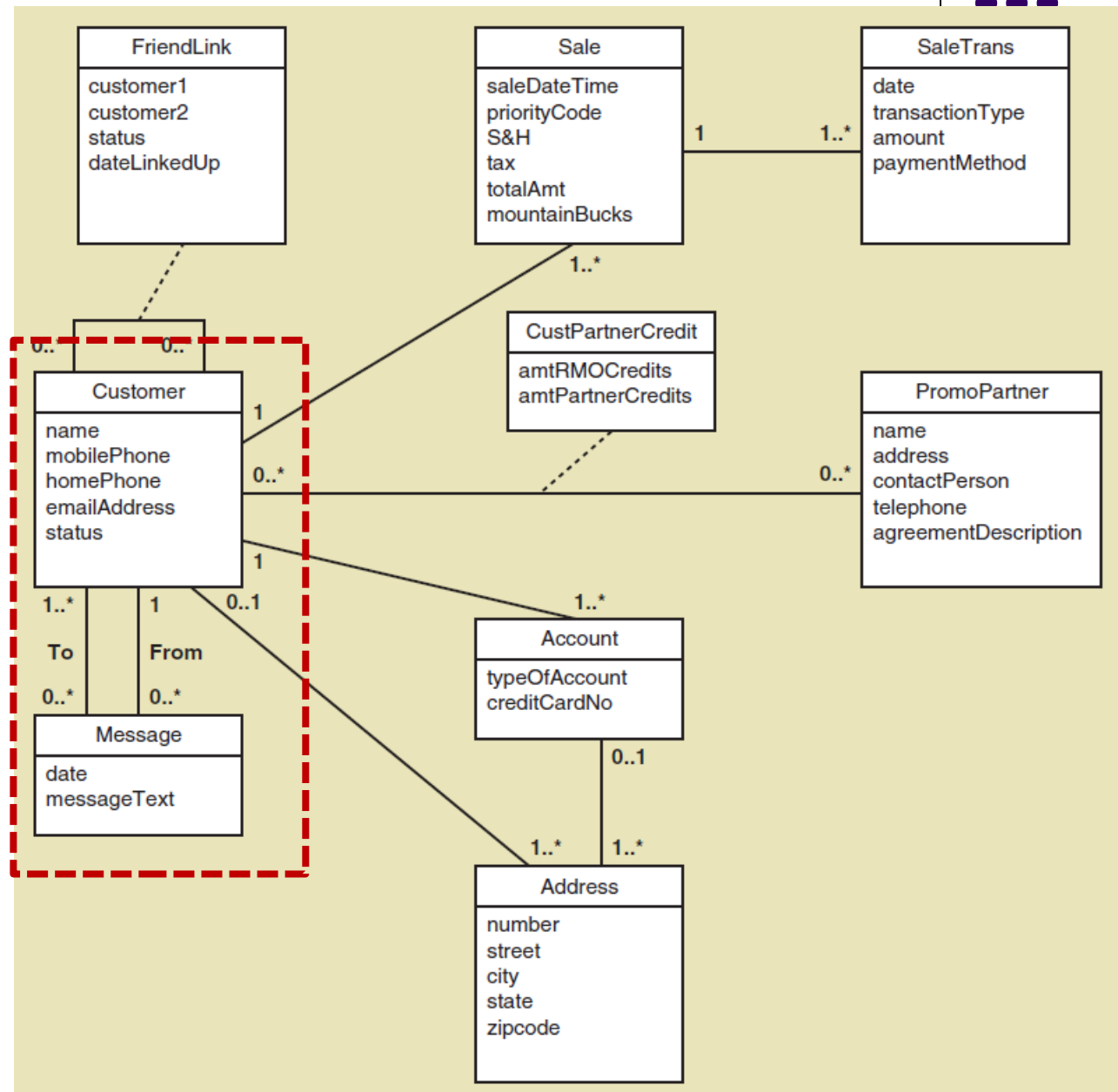




# RMO CSMS Project Customer Account Subsystem Domain Model Class Diagram



# RMO CSMS Project Customer Account Subsystem Domain Model Class Diagram



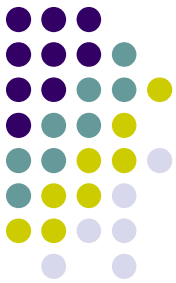
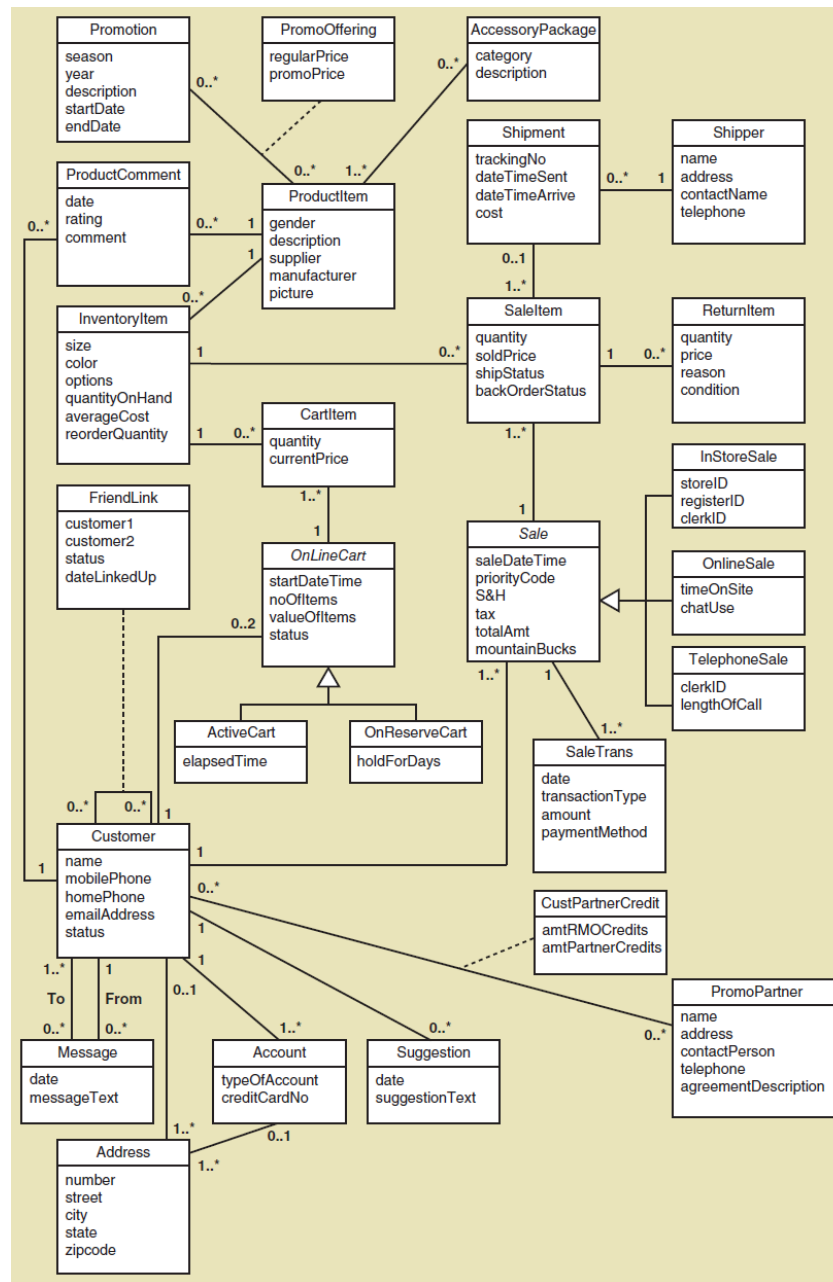
# RMO CSMS

## Project

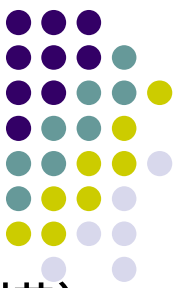
### Complete Domain

### Model Class

### Diagram

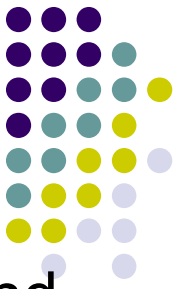


# Summary



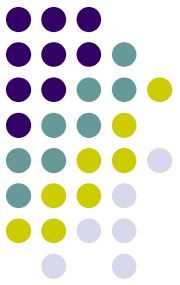
- This chapter is the second of three that focuses on modeling functional requirements (功能需求建模) as a part of systems analysis
- “Things” in the problem domain are identified and modeled, called domain classes or data entities (问题域中的“事物”被识别并建模)
- Two techniques for identifying domain classes/data entities are the brainstorming technique (头脑风暴) and the noun technique (名词技术)
- Domain classes have attributes (属性) and associations (关联)

# Summary



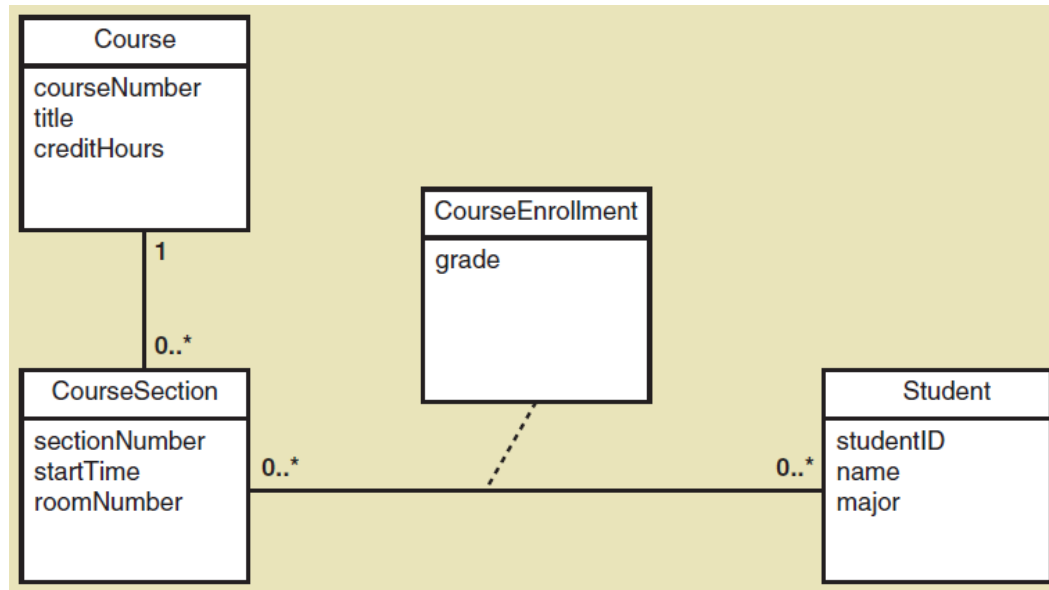
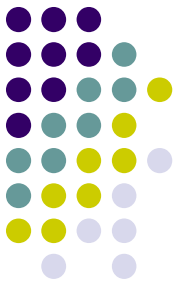
- Associations are naturally occurring relationships among classes, and associations have minimum and maximum multiplicity （关联是类之间自然存在的关系，并且关联具有最小和最大基数）
- The UML class diagram notation is used to create a domain model class diagram for a system. The domain model classes do not have methods because they are not yet software classes.
- There are actually three UML class diagram relationships: association relationships, generalization/specialization (inheritance) relationships, and whole part relationships （三种类图关系：关联关系，泛化 - 特化，整体部分）

# Summary



- Entity-relationship diagrams (ERDs) show the same information as a domain model class diagram (实体关系图 (erd) 显示与领域模型类图相同的信息)
- ERDs are preferred by database analysts and are widely used (erd 是数据库分析人员的首选，并且被广泛使用)
- ERDs are not UML diagrams, and an association is called a relationship, multiplicity is called cardinality, and generalization/specialization (inheritance) and whole part relationships are usually not shown

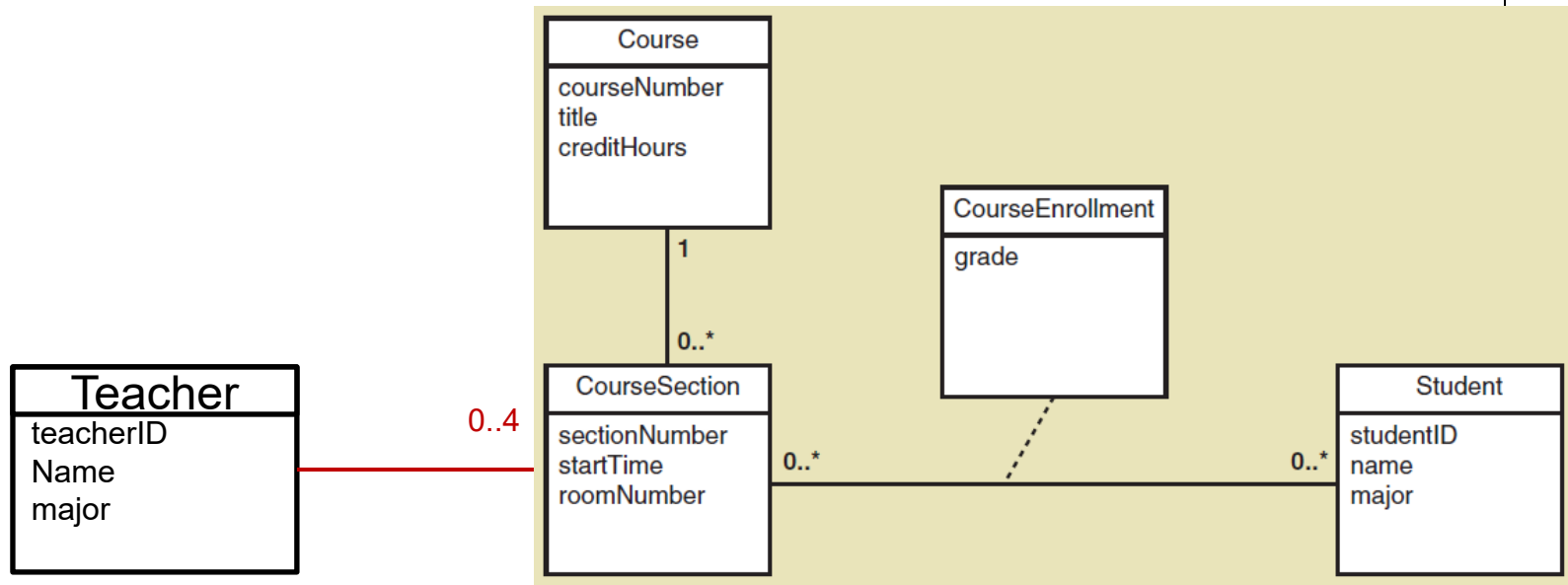
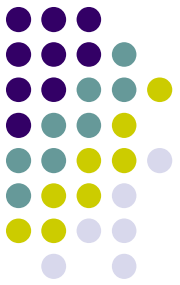
# 课后习题



- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。每个课程部分必须至少有一个教师来教授。同时，每个课程需要安排一位教师作为课程协调者来监督这个课程。

# 课后习题

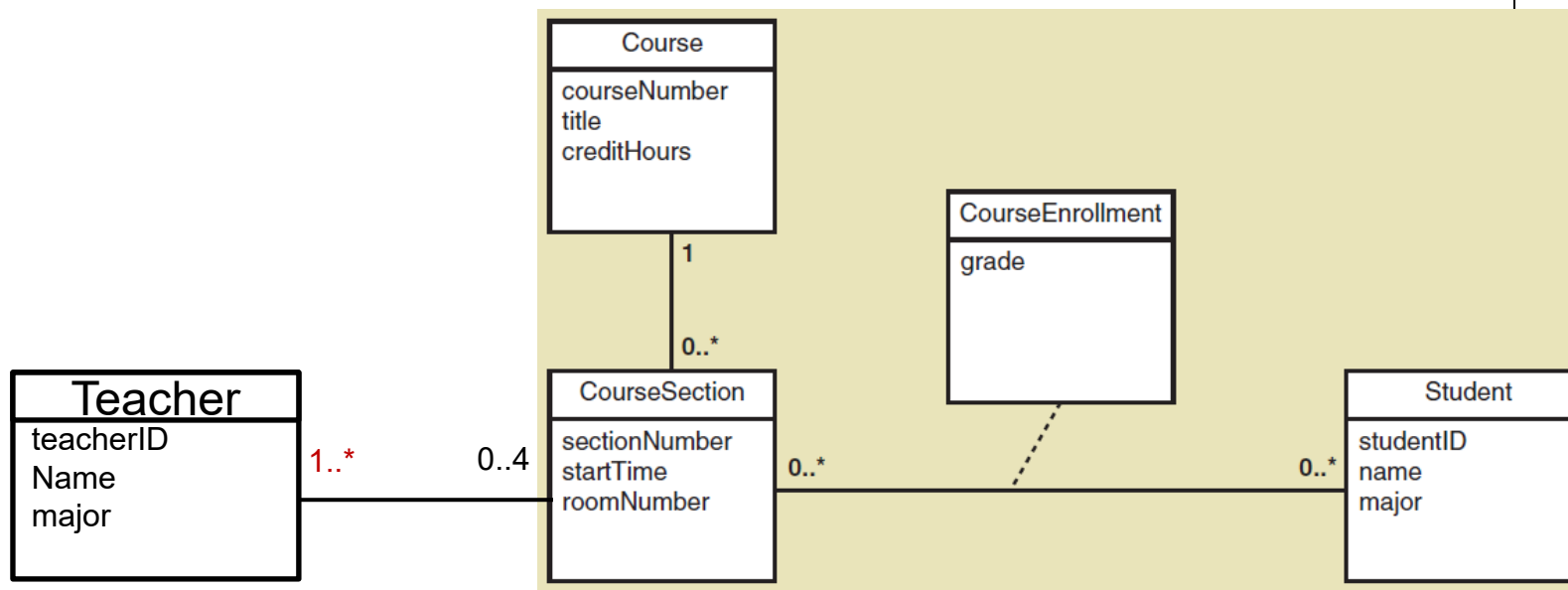


- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。每个课程部分必须至少有一个教师来教授。同时，每个课程需要安排一位教师作为课程协调者来监督这个课程。



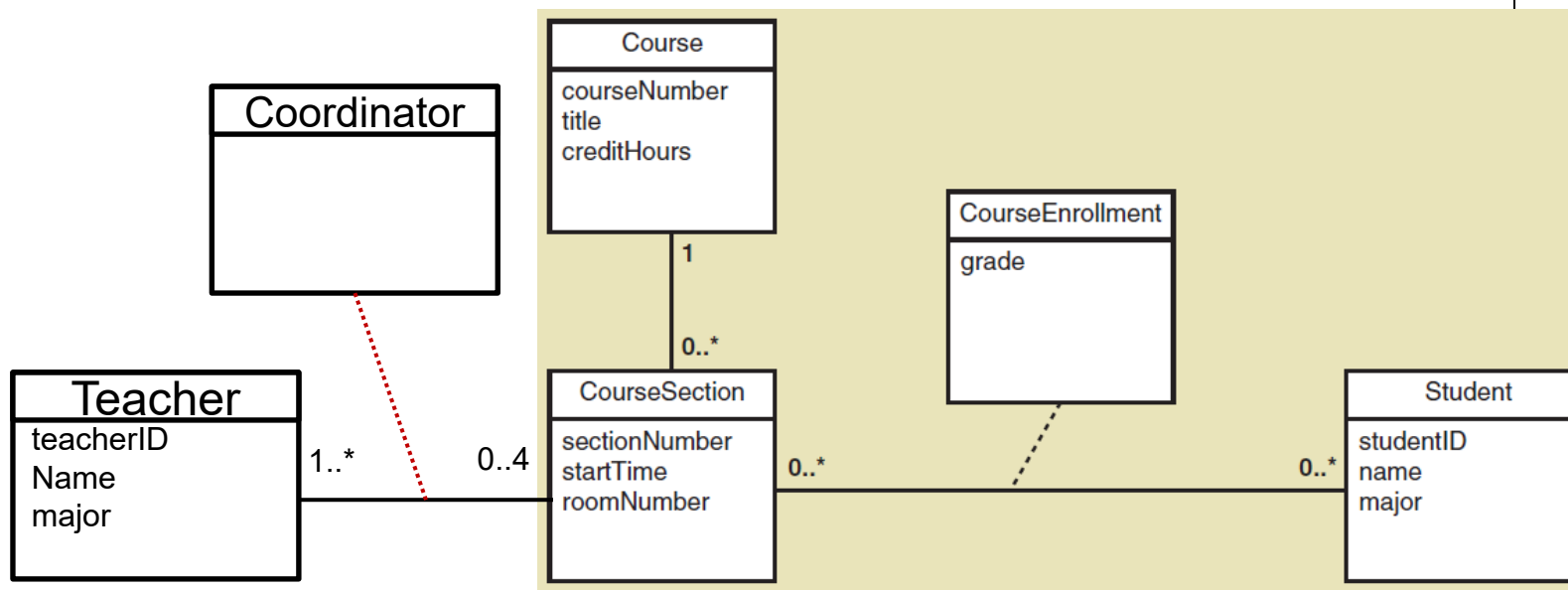
# 课后习题



- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。**每个课程部分必须至少有一个教师来教授。**同时，每个课程需要安排一位教师作为课程协调者来监督这个课程。

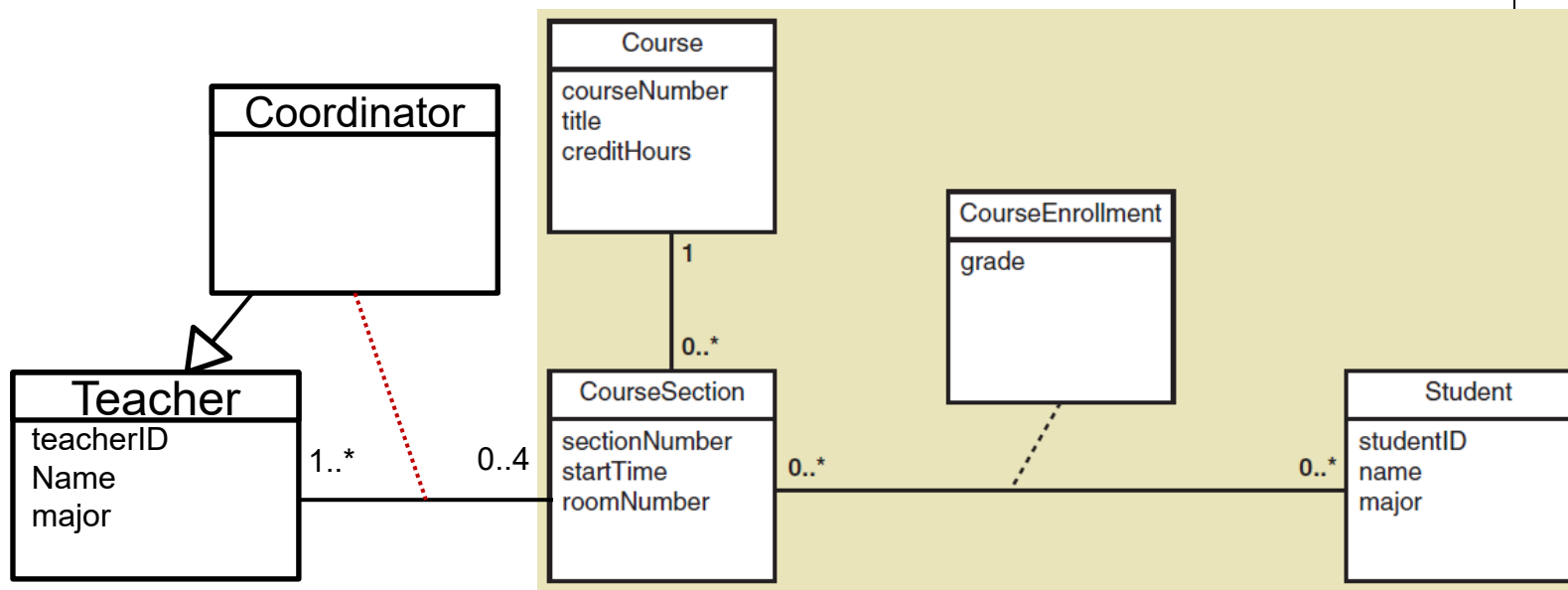
# 课后习题



- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。每个课程部分必须至少有一个教师来教授。同时，**每个课程需要安排一位教师作为课程协调者来监督这个课程。**

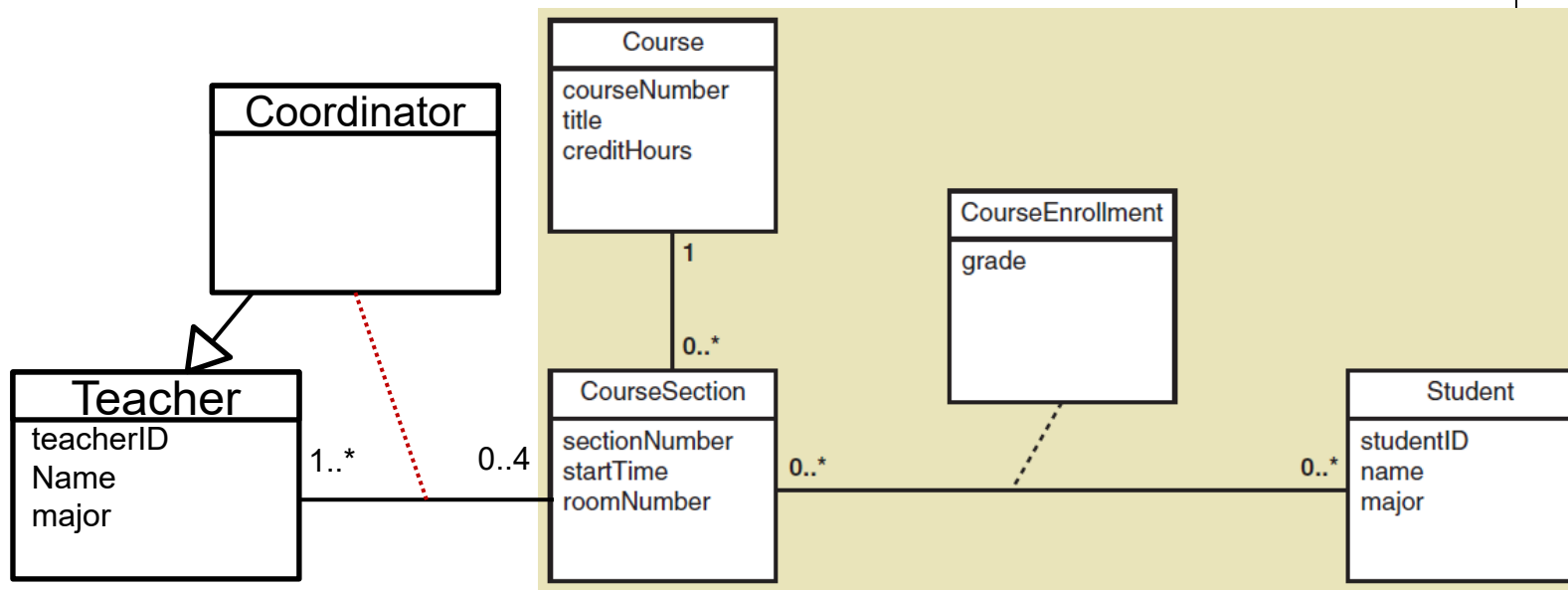
# 课后习题



- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。每个课程部分必须至少有一个教师来教授。同时，**每个课程需要安排一位教师作为课程协调者来监督这个课程。**

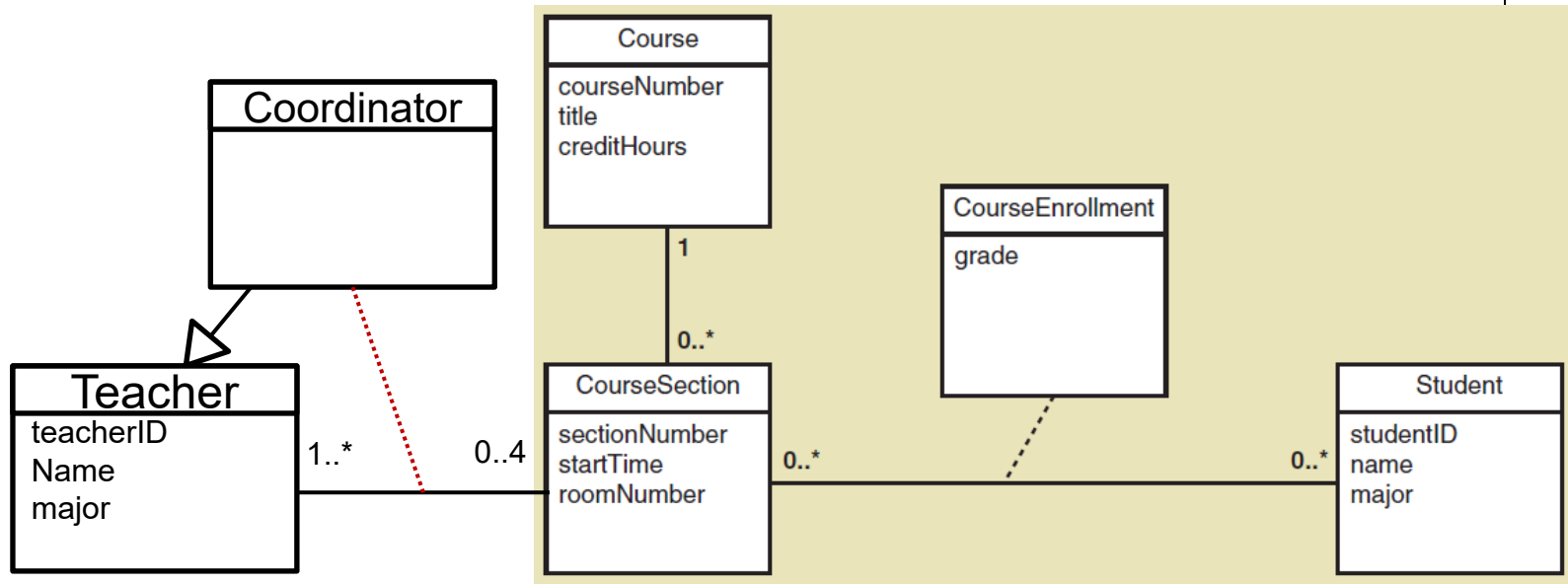
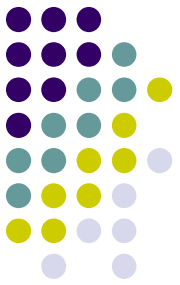
# 课后习题



- 再次思考上图，添加以下内容：

一个教师经常教授很多课程部分，但为了保证授课质量，一个教师一学期最多教授 4 门课。每个课程部分必须至少有一个教师来教授。同时，每个课程需要安排一位教师作为课程协调者来监督这个课程。

# 课后习题



- 课后思考：

添加后的域模型类图显示了教师与课程部分之间多对多的关系，那么更进一步地观察这个关系可能会揭露存储一些附加信息的需求。这样的需求包括：

每个课程部分的教师有特定的工作时间吗？

你会给每个课程部分的指导者一些评估吗？

...